<객체지향언어실습 기말 팀 프로젝트 계획서>

프로젝트 명 : 다이스 POS

(보드게임 카페 관리 프로그램)

— 피드백 반영 수정본 —

컴퓨터공학과 2 학년 3 반

팀원 : 김재민 (2020081053)
송하안 (2025109056)
김은별 (2025104899)
박준영 (2024081106)
서윤채 (2025101273)
김하진 (2025105269)

제출일 : 2026 년 11 월 3 일 (화요일)
수정 제출일 : 2026 년 11 월 9 일 (월요일)

피드백 반영 내역

교수님께서 남겨 주신 메모(전체 피드백 2 건, 페이지별 메모 16 건)를 모두 확인하고 아래와 같이 고쳤다. 오른쪽 칸의 위치에서 바뀐 내용을 볼 수 있다. 가장 중요하다고 하신 **결제 테이블**과 **직원 관리 목표**를 먼저 정한 뒤, 그 테이블을 쓰는 모든 팀원의 화면과 DB 표에 같은 구조를 반영했다.

번호	위치	피드백 내용	반영 내용 (→ 반영 위치)
1	전체 (1)	결제가 발생했을 때 결제 테이블을 어떻게 구성해 저장할지 (가장 중요)	결제를 **payment(결제) + payment_detail(결제 상세)** 2 개 테이블로 바꿨다. 결제 1 건은 payment 1 줄, 결제한 항목(이용료·메뉴)마다 payment_detail 1 줄로 저장하고 카드·현금·포인트 결제 금액을 따로 저장한다. 같은 방식(헤더 + 상세)을 주문과 발주에도 적용했다. → 3.2 ② (저장 예시 포함), ①·⑤·⑥
2	전체 (2)	직원 관리를 급여 계산까지 할지, 누적 근무시간만 저장할지 목표를 정해 테이블 수정	목표를 **급여 계산까지**로 정했다. staff 에 권한·고용 형태·시급·월 기본급 등을 넣고, 출퇴근을 날마다 쌓는 **work_log(근무 기록)**과 월 급여를 확정해 저장하는 **salary(급여)** 테이블, 근무·급여 관리 화면을 추가했다. → 3.2 ①
3	1. 제목	직원용 POS 화면과 손님 키오스크 화면을 구분해서 정리	2.2 에 **화면 구분표**(직원 화면 / 관리자 화면 / 키오스크)를 넣고, 구조도·역할 분담표·화면 그림 제목에 [직원]·[관리자]·[키오스크]를 표시했다. → 2.2 (1), 2.3, 3.1
4	2.2	로그인이 없어도 관리자·직원 화면 보안이 필요, 일반 직원의 관리자 화면 접근 차단	근무자 확인(출근 시 아이디·비밀번호), 관리자 확인 팝업(role = 관리자만 통과), 자동 잠금, 3 회 실패 시 30 초 잠금, 비밀번호 SHA-256 저장, 키오스크 창 분리를 넣었다. → 2.2 (3), 3.2 ①
5	2.2	키오스크에서 손님이 좌석 선택도 하는지	**손님은 좌석을 고르지 않는다.** 좌석 배정은 직원이 POS 테이블 현황에서 하고, 키오스크는 설치할 때 관리자 비밀번호로 테이블 번호를 고정한다. 이유도 적었다. → 2.2 (4)
6	2.3	역할 분담을 화면 단위로 한 것이 맞는지	맞다. 화면 단위로 나눴고, “화면을 맡은 사람이 그 화면이 저장하는 테이블을 담당하고, 다른 사람의 테이블은 담당자의 DAO 메서드로만 바꾼다”는 분담 원칙을 적었다. → 2.3
7	2.3 표	담당 화면이 키오스크인지 직원용(관리자) POS 인지 표에 표시	역할 분담표의 담당 화면마다 [직원]·[관리자]·[키오스크]를 붙였다. → 2.3
8	2.3 표 (김재민)	매출 관리에는 payment 테이블을 써야 함	바뀐 payment 로 결제 건수·매출·결제 수단별 합계를, payment_detail 로 이용료/메뉴 비율과 메뉴별 판매 순위를 집계한다. → 3.2 ①
9	2.3 표 (김은별)	게임 추천을 저장할 추천 테이블 필요 (member 참조, 손님-게임 매칭)	**recommend(게임 추천 기록)** 테이블을 추가했다. 추천할 때마다 이용 건·회원·게임을 1 줄씩 저장하고, 지난 추천 표시와 많이 추천된 게임 통계에 쓴다. → 3.2 ③
10	2.3 표 (서윤채)	결제(송하안)·메뉴 관리(박준영)와 요금 설정은 어떻게 다른지	화면 이름을 **이용 요금 설정**으로 바꾸고 차이를 표로 정리했다. 이용 요금 설정은 테이블을 쓴 시간에 대한 요금 규칙, 메뉴 관리는 음식 판매가, 결제는 두 금액을 합쳐 실제로 받은 돈을 기록한다. → 2.3
11	3.2 ① 그림	직원 직급(정규·계약·시간제) 구분 필요, 근무시간을 누적할지 로그처럼 쌓을지	staff.emp_type(고용 형태)을 추가했다. 근무시간은 **로그처럼 날마다 work_log 에 쌓고**, 누적 시간은 SUM 으로 계산한다(이유는 3.2 ① 다.에 적음).
12	3.2 ① staff 표	직급 필드, 해당일 근무시간·누적 근무시간·급여 필드	emp_type, role, hourly_wage, monthly_pay, hire_date, contract_end, status 를 추가하고, 해당일 근무시간은 work_log.work_minutes, 월 누적·급여는 salary 에 저장한다. → 3.2 ①

번호	위치	피드백 내용	반영 내용 (→ 반영 위치)
13	3.2 ① payment 표	송하안의 payment 와 구조가 다름. 참조 테이블도 모든 필드를 쓰고 사용 필드 표시	**모든 팀원에게 적용**했다. 참조 테이블도 담당자 표와 똑같이 전체 필드를 적고, '사용' 칸에 조회/변경을 표시했다(같은 스키마 원본에서 표를 만들어 서로 다를 수 없게 함). → 3.2 전체
14	3.2 ② payment 표	10,000 원 내고 주스 2 개·라면 1 개를 사면 어떻게 저장? 상품명·개수는 어디에?	주신 필드 구성대로 결제·결제 상세를 만들고, usage_id(이중 결제 방지)와 earn_point(적립)를 더했다. 상품 번호와 개수는 payment_detail 에 저장되며, 실제 값이 들어간 **저장 예시 표**를 넣었다. → 3.2 ②
15	3.2 ③ game 표	원하는 게임이 모두 사용 중이면 예약 기능	**game_rental(게임 대여·예약)** 테이블과 POS **게임 대여·예약 현황** 화면을 추가했다. 키오스크에서 대여 신청·예약을 하고, 반납되면 가장 먼저 예약한 테이블을 호출한다. → 3.2 ③
16	3.2 ④ menu 표	보드게임도 추가 구입·새 게임 구입·폐기 관리가 필요하지 않은지	발주 상세에 품목 구분(메뉴 / 보드게임)을 넣어 **보드게임도 발주**하고, 입고하면 game.stock 이 늘어난다. 새 게임 등록과 폐기 처리는 ③ 게임 관리에서 한다. → 3.2 ④, ③
17	3.2 ④ purchase_order 표	발주할 때 단가와 발주 금액은?	발주를 **purchase_order + purchase_order_detail** 로 나누고 매입 단가(unit_cost), 품목별 발주 금액(amount), 총 발주 금액(total_amount)을 저장한다. → 3.2 ④
18	3.2 ④ orders 표	orders 테이블의 용도, 김하진의 orders 와 필드가 다름	용도는 "주문 기록이 있는 메뉴의 삭제를 막기 위한 확인"이다. 주문이 2 개 테이블로 바뀌어 order_detail.menu_id 로 확인하며, 전체 필드를 적었다. → 3.2 ④
19	3.2 ⑤ cafe_table 표	테이블 수·좌석 수를 입력·수정·삭제하는 화면 필요	관리자 **테이블 설정** 화면을 추가했다(입력·수정·삭제). 우리 카페는 룸형이라 정원은 공사 없이는 바뀌지 않으므로 정원 입력·수정은 처음 등록과 공사 때만 쓰고, 고장·수리 중인 룸은 **사용가능 / 사용불가**로 바꿔 손님이 배정되지 않게 한다. → 3.2 ⑤
20	3.2 ⑥ orders 표	한 주문에 메뉴가 여러 개일 수 있음	**orders(주문) + order_detail(주문 상세)**로 바꿨다. 한 번 주문하면 orders 1 줄, 메뉴마다 order_detail 1 줄. 저장 예시를 넣었다. → 3.2 ⑥
21	3.2 ⑥ menu 표	박준영의 menu 와 구조가 다름, 전체 필드 작성 후 사용 필드 표시	13 번과 같이 반영했다. → 3.2 ⑥
22	팀 자체 수정	(기능 추가에 따른 업무량 조정)	기능이 늘어난 ③ 김은별의 **키오스크 이용 현황**을 ⑥ 김하진에게 옮겼다. 주문 내역과 예상 요금을 보여 주는 화면이라 주문 테이블 담당과 맞다. → 2.3

1. 제목

다이스 POS — 보드게임 카페 관리 프로그램

보드게임 카페에서 테이블 이용료, 음식 주문, 결제, 보드게임 대여·예약, 메뉴·발주, 직원 근무·급여, 매출을 한 프로그램으로 관리한다. 카운터에서 직원이 쓰는 **POS 창(직원 화면·관리자 화면)**과 테이블마다 놓고 손님이 쓰는 **키오스크 창**으로 나뉘며(2.2 의 화면 구분표 참고), Java Swing 과 MySQL 로 만든다.

2. 작품 개요

2.1 작품의 목적 및 필요성

보드게임 카페는 일반 카페와 다르게 손님이 있었던 시간과 인원만큼 이용료를 받고, 음식값은 나갈 때 한 번에 계산한다(후불제). 그런데 일반 카페 POS 로는 테이블별 이용 시간을 계산할 수 없어서, 직원이 입장 시간을 따로 적어 두었다가 직접 계산하다 보면 실수가 생긴다. 또 처음 온 손님은 게임이 너무 많아서 무엇을 할지 몰라 직원에게 자주 물어보고, 인기 게임은 이미 다른 테이블에서 쓰고 있어 기다리는 일이 많다.

그래서 이 프로젝트에서는 입장부터 주문, 결제까지 한 프로그램에서 처리하고 이용 요금은 자동으로 계산하려고 한다. 결제는 결제 1 건과 결제한 항목들을 나눠 저장해서 무엇을 얼마에 팔았는지 나중에도 확인할 수 있게 한다. 손님은 키오스크에서 게임 추천, 대여 신청·예약, 음식 주문, 예상 요금 확인을 직접 해서 직원 일을 줄인다. 직원은 출퇴근 기록을 남기고 월 급여까지 계산하며, 회원·보드게임·메뉴·발주·매출을 데이터베이스로 관리한다. 마지막으로 수업에서 배운 Swing 화면, 이벤트 처리, 예외처리, 스레드, JDBC(트랜잭션 포함)를 실제 프로그램에 써 보는 것이 목표이다.

2.2 작품 설명

다이스 POS 는 카운터 PC 에서 쓰는 **POS 메인 창**과, 각 테이블에 두는 **키오스크 창**으로 이루어진다. 두 창은 같은 MySQL 데이터베이스(dicepos)를 쓰기 때문에 키오스크에서 주문하거나 게임을 예약하면 POS 화면에도 바로 나타난다. POS 메인 창은 다시 모든 직원이 쓰는 **직원 화면**과 관리자만 여는 **관리자 화면**으로 나뉜다.

(1) 화면 구분 — 직원용 POS / 관리자용 POS / 손님 키오스크

구분	쓰는 사람 · 장소	화면 (담당 팀원)	들어가는 방법
직원 화면 (POS)	근무 중인 모든 직원 카운터 PC	테이블 현황(⑤), 결제 팝업(②), 결제 내역 조회(②), 회원 관리(②), 손님 주문받기(⑥), 게임 대여·예약 현황(③)	POS 를 켜고 근무자 확인(아이디·비밀번호)을 하면 열린다. 이때 출근이 기록된다.
관리자 화면 (POS)	권한이 '관리자'인 직원 카운터 PC	직원 관리(①), 근무·급여 관리(①), 매출 관리(①), 게임 관리(③), 메뉴 관리(④), 발주 관리(④), 테이블 설정(⑤), 이용 요금 설정(⑤)	왼쪽 메뉴의 관리자 화면(자물쇠 표시)을 누르면 관리자 확인 팝업이 뜨고, 관리자 비밀번호가 맞아야 열린다.
키오스크 (손님)	손님 각 테이블	키오스크 시작·메인(③), 게임 추천·대여 신청·예약(③), 음식·음료 주문(⑥), 이용 현황(⑥)	설치할 때 직원이 테이블 번호를 지정해 두고, 손님은 입장 등록 후 바로 쓴다.

(2) 화면별 주요 기능

직원 화면 : 테이블 현황에서 손님 입장을 등록하고, 손님 주문받기에서 메뉴를 주문받는다. 손님이 나갈 때는 결제 팝업에서 이용료와 음식값을 합쳐 카드·현금·포인트를 나눠서 결제할 수 있고, 결제 내역 조회에서 지난 결제와 결제 상세(영수증)를 확인한다. 회원 관리에서는 회원을 등록·수정·삭제하고 이름이나 전화번호 뒤 4 자리로 검색한다. 게임 대여·예약 현황에서는 대여 중인 게임과 예약 대기를 보고, 반납을 처리하면 다음 예약 테이블을 호출한다.

관리자 화면 : 직원 관리(권한·고용 형태·시급), 근무·급여 관리(출퇴근 기록 수정, 월 급여 계산·확정), 매출 관리(일별·월별, 결제 수단별, 메뉴별), 게임 관리(등록·폐기), 메뉴 관리(품절 처리), 발주 관리(메뉴·보드게임 발주, 입고하면 재고 증가), 테이블 설정(룸 등록·정원, 사용가능/사용불가), 이용 요금 설정(시간 요금제)을 한다.

키오스크 : 손님이 직접 쓴다. 인원, 난이도, 시간, 장르를 고르면 지금 빌릴 수 있는 게임을 추천해 주고, 바로 대여를 신청하거나 모두 대여 중이면 예약할 수 있다. 메뉴는 장바구니에 담아 한 번에 주문하고, 이용 현황에서 지금까지의 이용 시간·주문 내역·예상 요금을 본다.

(3) 보안 기능 — 로그인 창 대신 '확인 팝업'으로 화면 보호

별도의 로그인 화면은 만들지 않지만, 아래 기능으로 직원 화면과 관리자 화면을 보호하고 일반 직원이 관리자 화면에 들어가지 못하게 한다. 구현은 ① 김재민이 맡는다(직원 정보 staff 사용).

보안 기능	내용
근무자 확인	POS 를 켜면 직원 화면이 잠겨 있고, 직원이 자기 아이디·비밀번호를 넣어야 열린다. 이때 출근 시각이 work_log 에 기록되므로 다른 사람이 대신 출근을 찍을 수 없고, 결제·주문에는 확인된 직원의 번호가 저장된다.
관리자 확인	관리자 메뉴에는 자물쇠가 표시되고, 누르면 '관리자 확인' 팝업이 뜬다. staff.role 이 '관리자'인 계정의 비밀번호가 맞아야 열리므로 일반 직원은 관리자 화면에 들어갈 수 없다.
자동 잠금	관리자 화면은 직원 화면으로 돌아가거나 5 분 동안 조작이 없으면 다시 잠긴다(javax.swing.Timer). 직원 화면도 [화면 잠금] 버튼이나 10 분 동안 조작이 없으면 잠기고, 근무 중인 직원의 비밀번호로 푼다.
입력 실패 제한	비밀번호를 3 번 틀리면 30 초 동안 입력할 수 없다.
비밀번호 보호	비밀번호는 SHA-256(MessageDigest)으로 바꾼 값만 저장하고, 화면에는 ●로만 보이며 직원 목록 표에도 나오지 않는다.
키오스크 분리	키오스크는 별도 창(KioskFrame)으로 실행되어 POS 메뉴가 아예 없다. 키오스크의 테이블 번호를 바꾸거나 키오스크를 끌 때는 관리자 비밀번호가 필요하다.

(4) 키오스크에서 좌석을 고르지 않는 이유

손님은 키오스크에서 좌석을 고르지 않는다. 키오스크는 테이블마다 한 대씩 고정해 두고, 설치할 때 직원이 관리자 비밀번호를 넣어 그 키오스크의 테이블 번호를 지정한다. 좌석 배정(입장 등록)은 직원이 POS 테이블 현황에서 인원과 좌석 수를 확인하고 한다. 손님이 좌석을 직접 고르면 다른 테이블로 주문이 들어가거나, 좌석 수보다 많은 인원이 앉거나, 이미 손님이 있는 자리를 고르는 문제를 막기 어렵기 때문이다. 그래서 키오스크에서는 자기 테이블에 대해서만 게임 추천·대여 신청·주문·이용 현황 확인을 할 수 있다.

(5) 매장 구성 — 룸형 보드게임 카페 (참고 사진)

카운터에는 POS(직원·관리자 화면)가 있고, 게임 진열장 안쪽으로 번호가 붙은 룸이 위아래 2 층으로 놓여 있다. 손님은 룸 단위로 입장하며, 룸마다 테이블 위에 키오스크가 1 대씩 고정되어 있다. 룸은 크기가 정해진 방이라 정원은 공사 없이는 바뀌지 않는다(3.2 ⑤ 테이블 설정의 부연 설명 참고).



[참고 사진 1] 매장 전경 — 게임 진열장, 카운터(POS), 보드게임 룸 입구



[참고 사진 2] 룸 복도 — 번호가 붙은 2 층 구조 룸



[참고 사진 3] 룸 내부 — 테이블 위 키오스크

※ 위 사진은 매장 구성을 설명하기 위한 참고 이미지이다.

2.3 팀원별 역할 분담

역할은 **화면 단위**로 나눴다. 한 화면의 UI, 이벤트 처리, DB 연동은 한 사람이 끝까지 맡고, 그 화면이 저장(INSERT·UPDATE·DELETE)하는 테이블은 그 사람이 담당한다. 다른 팀원의 테이블은 SELECT 만 하고, 꼭 바꿔야 할 때는 담당자가 만든 DAO 메서드를 불러서 바꾼다(예: 결제할 때 ⑤의 TableUsageDAO.finish()로 이용 상태를 '결제완료'로 변경). 6 명이 각자 화면 3~4 개와 DB 테이블 2~3 개를 맡고, 모든 팀원이 Swing 화면, 이벤트 처리, 예외처리, DB 연동(CRUD), 트랜잭션을 직접 구현하도록 나눴다. 담당 화면 앞의 [직원]·[관리자]·[키오스크]는 2.2 (1)의 화면 구분이다.

담당	개발자	담당 화면	DB 테이블	주요 구현 요소
①	김재민	[관리자] 직원 관리(정보·메모 팝업), 근무·급여 관리, 매출 관리 [직원] 근무자 확인·관리자 확인 팝업	staff, work_log, salary	JDialog 팝업, 비밀번호 암호화, 급여 계산, GROUP BY 집계
②	송하안	[직원] 회원 관리, 결제 팝업, 결제 내역 조회	member, payment, payment_detail	실시간 검색, 결제 트랜잭션(헤더 + 상세), 복합 결제

담당	개발자	담당 화면	DB 테이블	주요 구현 요소
③	김은별	[관리자] 게임 관리 [직원] 게임 대여·예약 현황 [키오스크] 시작·메인, 게임 추천·대여 신청·예약	game, recommend, game_rental	JSlider, CardLayout, 추천 기록, 반납·예약 호출 트랜잭션
④	박준영	[관리자] 메뉴 관리, 발주 관리(메뉴·보드게임)	menu, purchase_order, purchase_order_detail	JCheckBox 품절, 발주 헤더 + 상세, 입고 트랜잭션
⑤	서윤채	[직원] 테이블 현황 [관리자] 테이블 설정, 이용 요금 설정	cafe_table, table_usage, fee_setting	테이블 카드 동적 생성, 요금 계산 클래스
⑥	김하진	[직원] 손님 주문받기 [키오스크] 음식·음료 주문, 이용 현황	orders, order_detail	메뉴 버튼 동적 생성, 장바구니, 주문 트랜잭션, 갱신 스레드

※ 이번 수정에서 ③ 김은별에게 게임 대여·예약과 추천 기록이 늘어나서, 원래 ③이 맡던 **키오스크 이용 현황**을 ⑥ 김하진이 맡도록 옮겼다. 이용 현황은 주문 내역과 예상 요금을 보여 주는 화면이라 주문 테이블(orders, order_detail) 담당과 맞다.

⑤ 이용 요금 설정과 ④ 메뉴 관리·② 결제의 차이

세 화면 모두 '금액'을 다루지만 다루는 대상이 다르다. 그래서 ⑤의 화면 이름을 '요금 설정'에서 **'이용 요금 설정'**으로 바꿨다.

화면 (담당)	다루는 금액	저장 테이블
이용 요금 설정 (⑤)	테이블을 쓴 **시간에 대한 요금 규칙** — 기본 시간, 1 인 기본요금, 추가 단위(분), 1 인 추가요금, 적용 시작일. 금액 하나가 아니라 계산 규칙이며, FeeCalculator 가 이 규칙과 이용 시간·인원으로 이용료를 계산한다.	fee_setting
메뉴 관리 (④)	음식·음료 **한 개의 판매 가격**과 재고·품절.	menu.price
결제 (②)	위의 이용료와 메뉴 금액을 합친 뒤 **실제로 받은 돈**(카드·현금·포인트)과 결제한 항목을 기록한다. 요금이나 가격을 정하지는 않는다.	payment, payment_detail

평가표의 결과물 테스트 항목(컴포넌트, 배치, 이벤트 처리, DB 연동, 난이도)을 모두 자기 화면에서 보여 줄 수 있도록 팀원마다 구현할 내용을 정했다. 각자 자기 화면과 자기 테이블만 만들기 때문에 맡은 부분이 겹치지 않는다.

① 김재민 — 직원 관리 · 근무·급여 관리 · 매출 관리

평가 항목	구현 내용
컴포넌트	JTable, JDialog(정보·메모·확인 팝업), JTextField, JPasswordField, JTextArea, JComboBox(고용 형태·직원·월), JRadioButton(권한), JSpinner(추가 수당)
컨테이너·배치	BorderLayout, FlowLayout(버튼 줄), GridLayout(팝업 입력 폼), JSplitPane(근무 기록 / 급여 계산), JScrollPane
이벤트 처리	ActionListener(추가·변경·퇴사·메모·조회·급여 확정), KeyListener(이름·아이디 검색), ItemListener(고용 형태 → 시급/월급 칸 전환, 일별/월별), javax.swing.Timer(자동 잠금)
DB 연동	staff·work_log·salary INSERT / UPDATE / DELETE / SELECT, payment·payment_detail 기간별 GROUP BY 집계(COUNT, SUM)
예외처리·난이도	아이디 중복·전화번호 형식·최저시급 검사, 같은 달 급여 중복 확정 방지(UNIQUE), 비밀번호 SHA-256·3 회 실패 잠금, 매출 그래프 직접 그리기

② 송하안 — 회원 관리 · 결제 · 결제 내역 조회

평가 항목	구현 내용
컴포넌트	JTable, JTextField, JRadioButton, JComboBox, JSpinner(포인트·카드 금액), JLabel(합계 검증)

평가 항목	구현 내용
컨테이너·배치	BorderLayout, GridLayout(입력 폼), JDialog(결제 팝업), JSplitPane(결제 목록 / 결제 상세), JScrollPane
이벤트 처리	KeyListener(실시간 검색), MouseListener(표 클릭 → 상세 조회), ChangeListener(포인트·카드 금액 → 현금 금액 자동 계산)
DB 연동	member INSERT / UPDATE / DELETE / SELECT, payment·payment_detail INSERT / SELECT, table_usage UPDATE(⑤ DAO 호출)
예외처리·난이도	결제 트랜잭션(payment 1 줄 + payment_detail 여러 줄 + 포인트·등급 + 이용 상태, commit / rollback), 금액 합계 불일치·포인트 초과·이중 결제 방지

③ 김은별 — 게임 관리 · 게임 대여·예약 · 키오스크(시작·게임 추천)

평가 항목	구현 내용
컴포넌트	JTable, JSlider(난이도), JToggleButton + ButtonGroup, JLabel 결과 카드, JButton(대여 신청·예약·반납)
컨테이너·배치	BorderLayout, GridLayout(추천 카드), CardLayout(키오스크 화면 전환)
이벤트 처리	KeyListener(이름 검색), ChangeListener(슬라이더 → ★), MouseListener(서비스 카드), ActionListener(대여·반납·예약), Timer(5 초 대여 현황 갱신)
DB 연동	game INSERT / UPDATE / DELETE / SELECT, 조건 검색 SELECT, recommend INSERT / UPDATE / SELECT, game_rental INSERT / UPDATE / SELECT
예외처리·난이도	반납 + 다음 예약 호출 트랜잭션, 대여 가능 수량 계산(보유 − 대여 중), 중복 예약·대여 중인 게임 폐기 막기, 추천 결과 없음 안내

④ 박준영 — 메뉴 관리 · 발주 관리

평가 항목	구현 내용
컴포넌트	JTable, JCheckBox(품절), JComboBox(품목), JToggleButton(메뉴 / 보드게임), JSpinner, JTextField
컨테이너·배치	BorderLayout, GridBagLayout(메뉴 입력 폼), FlowLayout(발주 입력 줄), JScrollPane
이벤트 처리	ItemListener(품절 체크박스, 품목 구분 → 품목 목록 바꾸기), ListSelectionListener(발주 선택 → 상세 표·버튼 활성), ActionListener(담기·발주·입고)
DB 연동	menu INSERT / UPDATE / DELETE / SELECT, purchase_order·purchase_order_detail INSERT / UPDATE / SELECT, game 보유 수량 UPDATE(③ DAO 호출)
예외처리·난이도	발주 저장 트랜잭션(헤더 + 상세), 입고 트랜잭션(발주 상태 + 메뉴 재고 + 게임 보유 수량), 재고 0 이면 자동 품절, 주문 기록 있는 메뉴 삭제 막기

⑤ 서윤채 — 테이블 현황 · 테이블 설정 · 이용 요금 설정

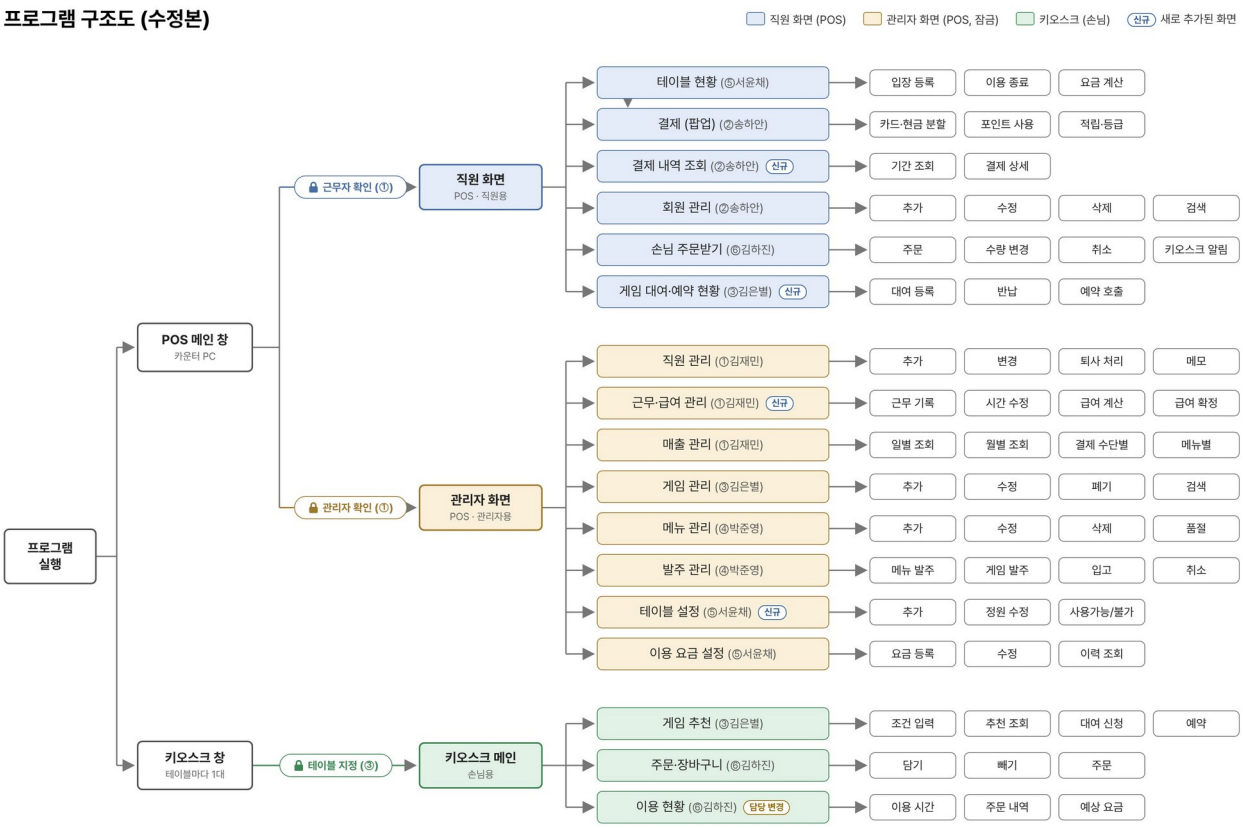
평가 항목	구현 내용
컴포넌트	직접 만든 테이블 카드(JPanel), JSpinner, JTextField, JCheckBox(사용 여부), JTable
컨테이너·배치	GridLayout(테이블 수에 맞춰 행 계산), BorderLayout, JDialog(이용 시작·종료 팝업)
이벤트 처리	MouseListener(카드 클릭), ActionListener(이용 시작·종료·요금 계산, 테이블 추가·수정·중지), 표 선택
DB 연동	cafe_table·table_usage·fee_setting INSERT / UPDATE / DELETE / SELECT, member·game_rental SELECT
예외처리·난이도	1 분 갱신 스레드, 요금 계산 클래스 FeeCalculator(②·⑥ 공용), 좌석 수 초과 검사, 사용불가 룸 입장 차단, 이용 기록 있는 룸은 삭제 대신 사용불가

⑥ 김하진 — 손님 주문받기(POS) · 키오스크 주문 · 이용 현황

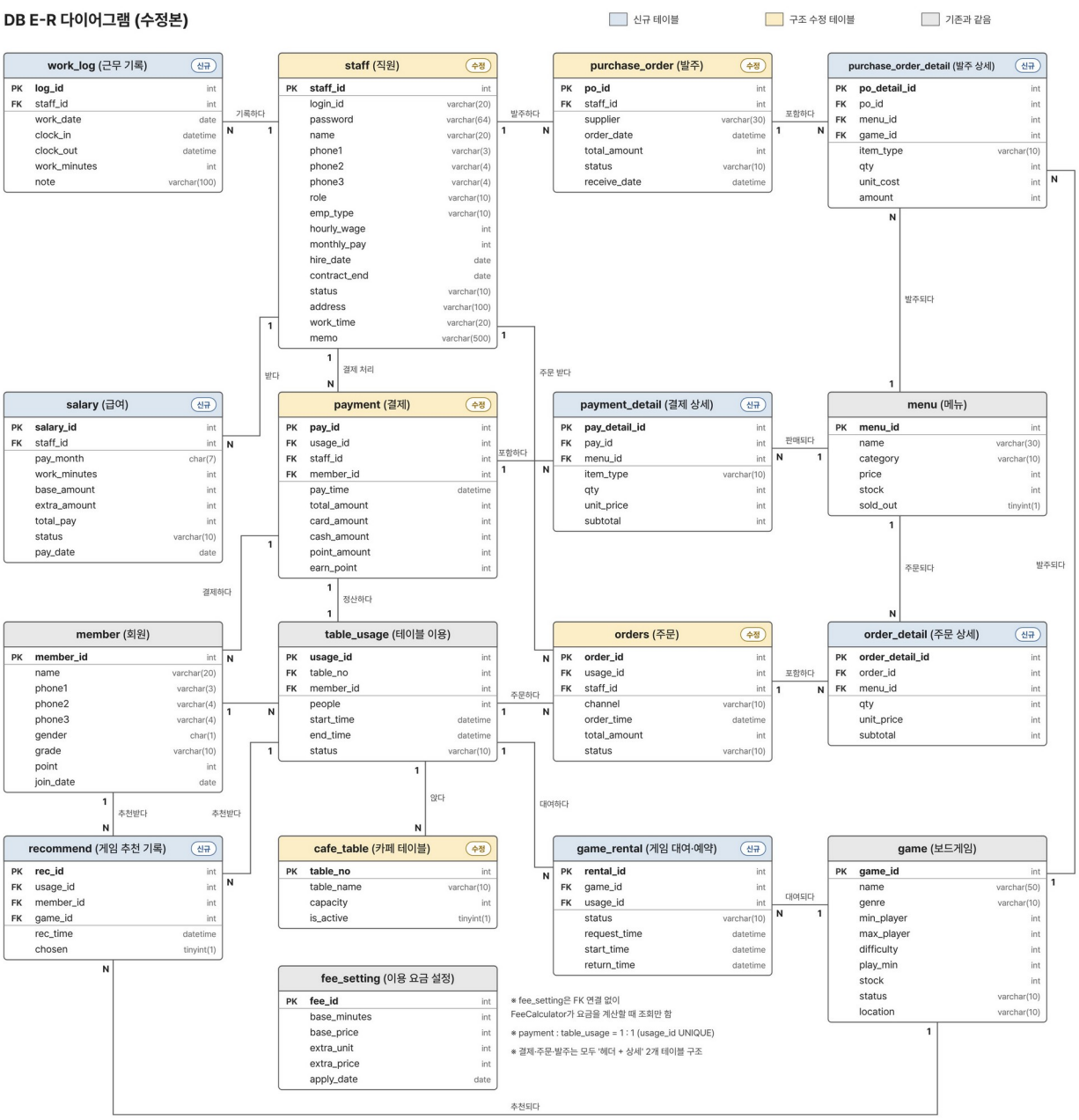
평가 항목	구현 내용
컴포넌트	메뉴 JButton 동적 생성, 직접 만든 MenuCard, JList(장바구니), JTable, JComboBox, JLabel(이용 현황)
컨테이너·배치	GridLayout(메뉴 버튼·카드), BorderLayout, JTabbedPane(주문정보·이용정보), JScrollPane
이벤트 처리	리스너 하나를 모든 메뉴 버튼에 등록, ItemListener(테이블 선택), Timer(5 초 키오스크 주문 알림), Thread(30 초 이용 현황 갱신)
DB 연동	orders·order_detail INSERT / UPDATE / DELETE / SELECT, menu SELECT·재고 UPDATE(④ DAO 호출), table_usage SELECT
예외처리·난이도	주문 트랜잭션(orders 1 줄 + order_detail 여러 줄 + 메뉴 재고 감소), 같은 메뉴 수량 합치기, 품절 메뉴 비활성화, 이용 중이 아닌 테이블 안내

3. 작품 내용

3.1 전체 프로그램 구조도



[그림 1] 전체 프로그램 구조도 (괄호 안은 담당 팀원, 색은 화면 구분, 자물쇠는 확인 팝업)



[그림 2] 전체 DB E-R 다이어그램 (17 개 테이블)

DB 설계에서 바뀐 점 : 결제·주문·발주처럼 '한 번에 여러 품목'이 생기는 데이터는 모두 **헤더 + 상세** 2 개 테이블로 나눴다(payment / payment_detail, orders / order_detail, purchase_order / purchase_order_detail). 헤더에는 1 건 전체의 정보(일시, 직원, 회원, 총액 등)를, 상세에는 품목마다 번호·수량·단가·소계를 한 줄씩 저장한다. 이 밖에 직원 근무·급여(work_log, salary), 게임 추천 기록(recommend), 게임 대여·예약(game_rental)을 새로 만들어 테이블이 10 개에서 17 개가 되었다.

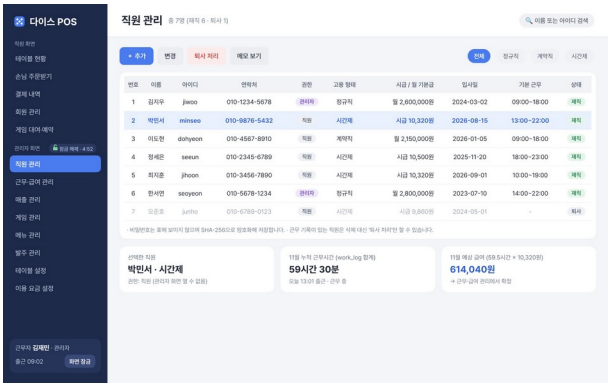
3.2 화면별 설명

팀원마다 같은 순서(가. 화면 그림 → 나. 기능 설명 → 다. 연동 DB 구조)로 정리했다. 화면 이름 뒤의 [직원]·[관리자]·[키오스크]는 화면 구분이다. DB 표에서 PK 는 기본키, FK 는 외래키(→ 참조하는 테이블.필드)이다. 다른 팀원이 담당하는 테이블은 '☑️ 참조'로 표시하고, **그 테이블의 모든 필드를 담당자 표와 똑같이 적은 뒤 '사용' 칸에 자기가 쓰는 필드를 표시**했다(조회 = SELECT, 변경 = 담당자의 DAO 메서드로 UPDATE). 회색 글씨는 쓰지 않는 필드이다.

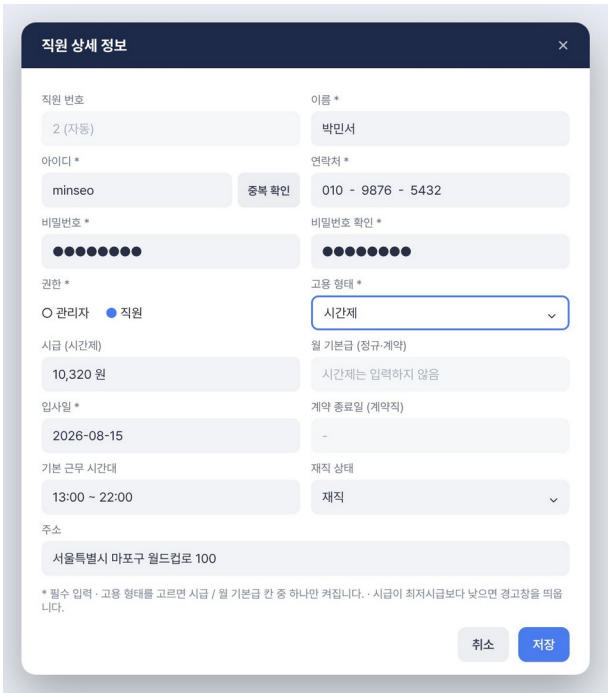
① 개발자 명 : 김재민 (학번 : 2020081053) — 직원 관리 · 근무·급여 관리 · 매출 관리

담당 테이블 : staff, work_log, salary / 참조 테이블 : payment, payment_detail, menu

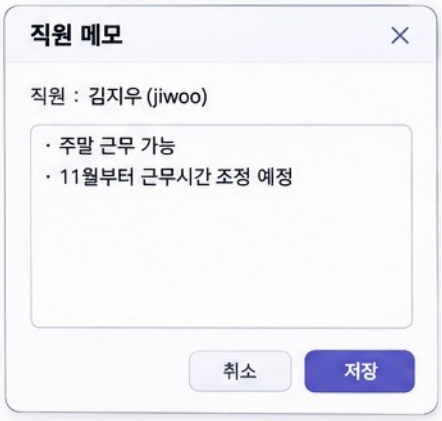
가. 개발할 화면 그림



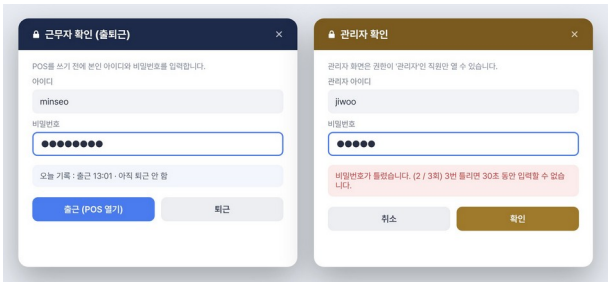
[그림 ①-1] 직원 관리 [관리자]



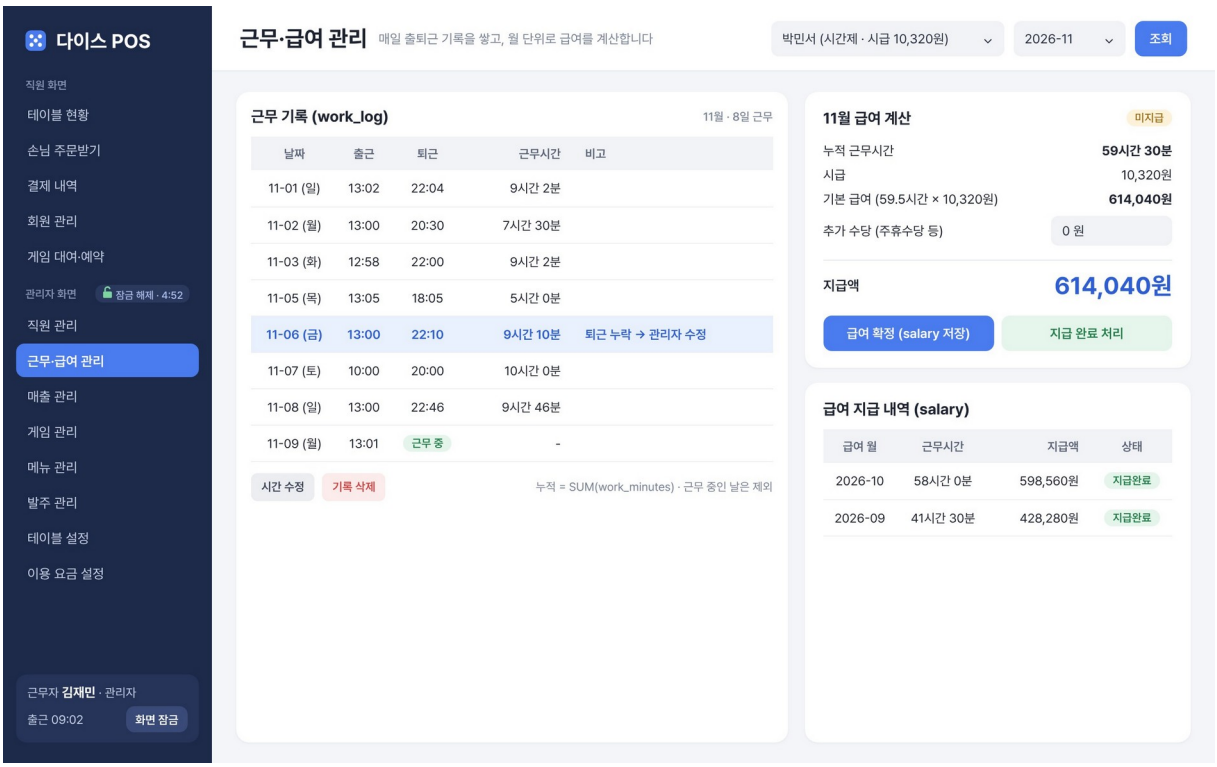
[그림 ①-2] 직원 상세 정보 팝업 [관리자]



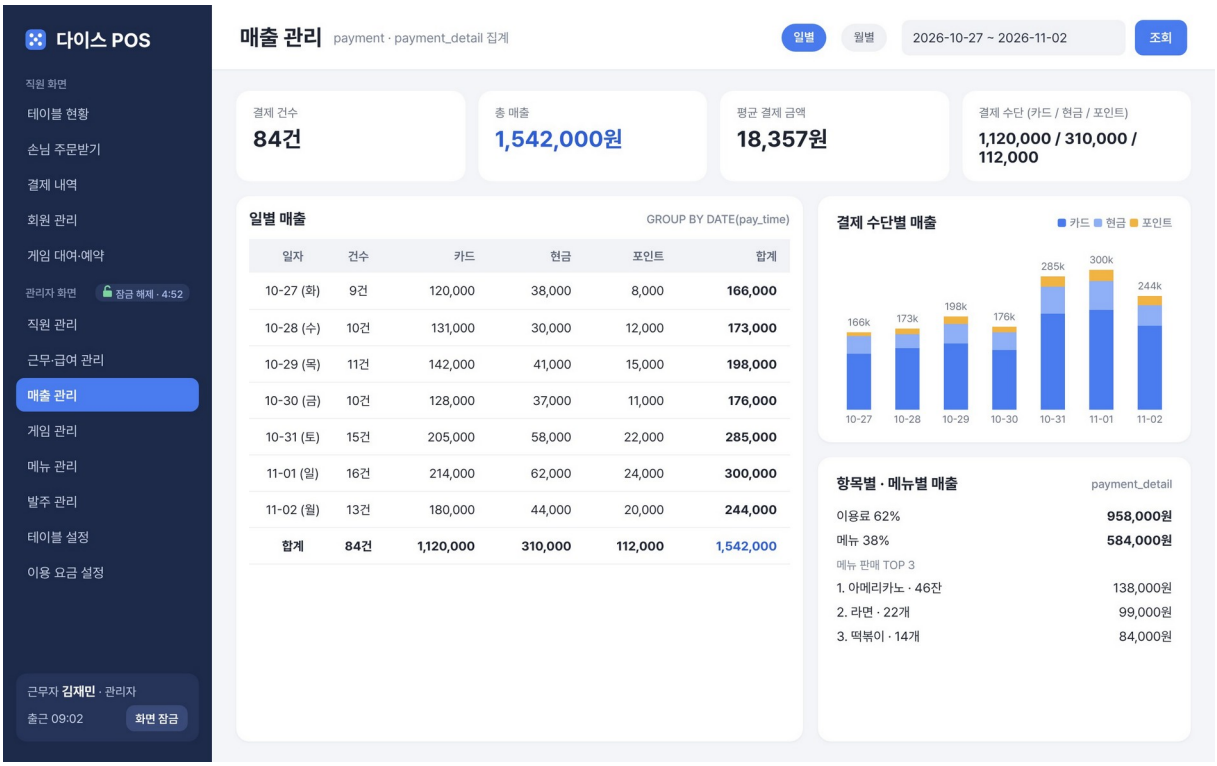
[그림 ①-3] 직원 메모 팝업 [관리자]



[그림 ①-4] 근무자 확인 · 관리자 확인 팝업 [직원]



[그림 ①-5] 근무·급여 관리 [관리자] — 날마다 쌓은 근무 기록과 월 급여 계산



[그림 ①-6] 매출 관리 [관리자] (일별 조회)

나. 화면 기능 설명

화면	버튼 / 이벤트	동작 (DB 작업 포함)
직원 관리	화면 열림 / 고용 형태 버튼	staff 를 SELECT 해서 번호, 이름, 아이디, 연락처, 권한, 고용 형태, 시급·월 기본급, 입사일, 상태를 JTable 에 보여 준다. 정규직·계약직·시간제 버튼을 누르면 그 형태만 다시 조회한다.
직원 관리	검색창 입력 (KeyListener)	이름이나 아이디에 검색어가 들어간 직원만 다시 조회한다.

화면	버튼 / 이벤트	동작 (DB 작업 포함)
직원 관리	[추가] / [변경] 버튼	직원 상세 정보 팝업(JDialog)을 연다. 고용 형태를 고르면 시간제는 시급 칸만, 정규직·계약직은 월 기본급 칸만 켜진다(계약직은 계약 종료일도). 필수값·전화번호 숫자 형식·아이디 중복·비밀번호 확인 일치·최저시급을 검사한 뒤 비밀번호를 SHA-256 으로 바꿔 INSERT 또는 UPDATE 한다.
직원 관리	[퇴사 처리] 버튼	근무 기록(work_log)이 없는 직원은 확인창 후 DELETE 하고, 기록이 있으면 근무·급여 기록을 지키기 위해 status 만 '퇴사'로 UPDATE 한다. 퇴사한 직원은 근무자 확인을 할 수 없다.
직원 관리	[메모 보기] / 저장	고른 직원의 memo 를 메모 팝업(JTextArea)에 보여 주고, 저장을 누르면 memo 를 UPDATE 한다.
근무자 확인 [직원]	POS 시작 · [출근] / [퇴근]	아이디·비밀번호를 SHA-256 으로 바꿔 staff 와 비교한다. 맞으면 출근은 work_log 에 INSERT(work_date, clock_in) 하고 직원 화면을 연다. 퇴근은 clock_out 과 work_minutes(퇴근 − 출근, 분)를 UPDATE 한다. 이미 출근한 상태에서 다시 출근하면 안내창을 띄운다.
관리자 확인 [직원]	관리자 메뉴 클릭	관리자 확인 팝업을 띄워 role 이 '관리자'인 계정의 비밀번호인지 확인한다. 3 번 틀리면 30 초 동안 막고, 열린 뒤 5 분 동안 조작이 없으면 Timer 로 다시 잠근다.
근무·급여 관리	직원·월 선택 후 [조회]	고른 직원의 그 달 work_log 를 SELECT 해서 날짜별 출근·퇴근·근무시간을 보여 주고, SUM(work_minutes)으로 누적 근무시간을 계산한다.
근무·급여 관리	[시간 수정] / [기록 삭제]	퇴근을 찍지 않은 날 등은 관리자가 시간을 고치고 사유를 note 에 남긴다(UPDATE). 퇴근 시각이 출근보다 빠르면 경고창을 띄운다. 이미 확정된 달의 기록은 고칠 수 없다.
근무·급여 관리	급여 계산 / [급여 확정] / [지급 완료 처리]	시간제는 누적 시간 × 시급, 정규직·계약직은 월 기본급으로 기본 급여를 계산하고 추가수당(주휴수당 등)을 더해 salary 에 INSERT 한다. 같은 직원·같은 달은 한 번만 확정할 수 있다(UNIQUE). [지급 완료 처리]를 누르면 status 와 pay_date 를 UPDATE 한다.
매출 관리	일별 / 월별 조회 버튼	payment 를 날짜별 또는 월별로 GROUP BY 해서 결제 건수(COUNT), 매출(SUM total_amount), 카드·현금·포인트 합계를 표와 막대그래프로 보여 준다.
매출 관리	기간 선택 후 [조회]	시작일 ~ 종료일 사이의 결제만 다시 조회하고 결제 건수·총 매출·평균 결제 금액을 위쪽 카드에 표시한다. payment_detail 을 item_type 별, menu_id 별로 합계 내서 이용료 / 메뉴 비율과 메뉴 판매 순위(menu 와 JOIN)를 보여 준다.

다. 연동 데이터베이스 구조

직원 관리 목표 : 급여 계산까지 한다. 직원 정보(staff)에 권한·고용 형태·시급·월 기본급을 넣고, 근무시간은 날마다 근무 기록(work_log)에 쌓으며, 월 급여는 계산 결과를 급여(salary)에 확정해 저장한다.

테이블 : staff (직원) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
staff_id	INT (AUTO_INCREMENT)	PK	직원 번호
login_id	VARCHAR(20)	UNIQUE	아이디 (중복 불가)
password	VARCHAR(64)		비밀번호 (SHA-256 으로 암호화한 값만 저장)
name	VARCHAR(20)		이름
phone1 ~ phone3	VARCHAR(3) / (4) / (4)		전화번호 (앞 / 가운데 / 끝자리)
role	VARCHAR(10)		권한 (관리자 / 직원). 관리자만 관리자 화면을 열 수 있음
emp_type	VARCHAR(10)		고용 형태 (정규직 / 계약직 / 시간제)

필드명	필드 자료형	PK / FK	필드 설명
hourly_wage	INT		시급 (시간제만 입력, 나머지는 NULL)
monthly_pay	INT		월 기본급 (정규직·계약직만 입력)
hire_date	DATE		입사일
contract_end	DATE		계약 종료일 (계약직만 입력)
status	VARCHAR(10)		재직 상태 (재직 / 퇴사)
address	VARCHAR(100)		주소 (선택 입력)
work_time	VARCHAR(20)		기본 근무 시간대 (예: 09:00~18:00, 선택 입력)
memo	VARCHAR(500)		직원 메모 (선택 입력)

테이블 : work_log (근무 기록) ☑️ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
log_id	INT (AUTO_INCREMENT)	PK	근무 기록 번호
staff_id	INT	FK → staff.staff_id	근무한 직원
work_date	DATE		근무 날짜
clock_in	DATETIME		출근 시각
clock_out	DATETIME		퇴근 시각 (근무 중이면 NULL)
work_minutes	INT		그날 근무시간(분). 퇴근할 때 계산해서 저장
note	VARCHAR(100)		비고 (관리자가 시간을 고친 경우 그 사유)

테이블 : salary (급여) ☑️ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
salary_id	INT (AUTO_INCREMENT)	PK	급여 번호
staff_id	INT	FK → staff.staff_id	급여를 받는 직원
pay_month	CHAR(7)	UNIQUE (staff_id 와 묶음)	급여 월 (예: 2026-11). 같은 달 중복 확정 불가
work_minutes	INT		그 달 누적 근무시간(분) = work_log 합계
base_amount	INT		기본 급여 (시간제: 누적 시간 × 시급, 정규·계약: 월 기본급)
extra_amount	INT		추가 수당 (주휴수당 등, 관리자 입력, 기본값 0)
total_pay	INT		지급액 (base_amount + extra_amount)
status	VARCHAR(10)		지급 상태 (미지급 / 지급완료)
pay_date	DATE		지급일 (지급 전이면 NULL)

근무시간을 '누적값 하나'가 아니라 '날마다 쌓는 기록'으로 저장한 이유

- 근무시간은 날마다 다르고, 퇴근을 찍지 않은 날처럼 잘못된 기록을 그날만 골라 고칠 수 있어야 한다. 누적값 하나만 있으면 어느 날이 틀렸는지 알 수 없다.
- 누적 근무시간은 SUM(work_minutes)으로 원하는 기간(한 주, 한 달)만큼 언제든 계산할 수 있다. 누적값을 따로 저장하면 기록과 값이 서로 어긋날 위험만 생긴다.
- 월 급여는 확정한 결과를 salary 에 저장하므로, 나중에 시급이 바뀌어도 지난달 지급액은 바뀌지 않는다.

테이블 : payment (결제) ☐ 담당 ☑ 참조 (담당: ② 송하안)

필드명	필드 자료형	PK / FK	필드 설명	사용
pay_id	INT (AUTO_INCREMENT)	PK	결제 번호	조회
usage_id	INT	FK → table_usage.usage_id (UNIQUE)	결제한 테이블 이용 건 (한 이용 건은 한 번만 결제)	
staff_id	INT	FK → staff.staff_id	결제를 처리한 직원	
member_id	INT	FK → member.member_id	결제한 회원 (비회원은 NULL)	
pay_time	DATETIME		결제 일시 (연월일 시분초)	조회
total_amount	INT		총 결제 금액 (= 결제 상세 소계의 합)	조회
card_amount	INT		카드 결제 금액	조회
cash_amount	INT		현금 결제 금액	조회
point_amount	INT		포인트 결제 금액 (사용한 포인트)	조회
earn_point	INT		이번 결제로 적립한 포인트 (비회원은 0)	

테이블 : payment_detail (결제 상세) ☐ 담당 ☑ 참조 (담당: ② 송하안)

필드명	필드 자료형	PK / FK	필드 설명	사용
pay_detail_id	INT (AUTO_INCREMENT)	PK	결제 상세 번호	
pay_id	INT	FK → payment.pay_id	어느 결제의 상세인지	조회
item_type	VARCHAR(10)		항목 구분 (이용료 / 메뉴)	조회
menu_id	INT	FK → menu.menu_id	상품(메뉴) 번호. 이용료 줄은 NULL	조회
qty	INT		구매 수량 (이용료 줄은 이용 인원)	조회
unit_price	INT		구매 단가 (메뉴는 menu.price 에서 불러옴, 이용료는 1 인 요금)	
subtotal	INT		상품별 결제 금액 소계 (qty × unit_price)	조회

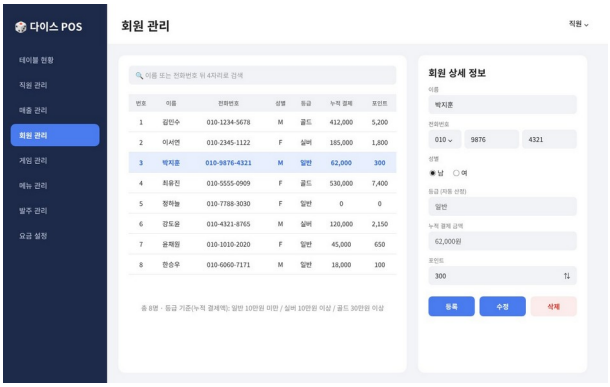
테이블 : menu (메뉴) ☐ 담당 ☑ 참조 (담당: ④ 박준영)

필드명	필드 자료형	PK / FK	필드 설명	사용
menu_id	INT (AUTO_INCREMENT)	PK	메뉴 번호	조회
name	VARCHAR(30)	UNIQUE	메뉴 이름 (중복 불가)	조회
category	VARCHAR(10)		분류 (음료 / 간식 / 식사)	
price	INT		판매 가격	
stock	INT		재고 수량, 기본값 0	
sold_out	TINYINT(1)		품절 여부 (1 이면 주문 불가)	

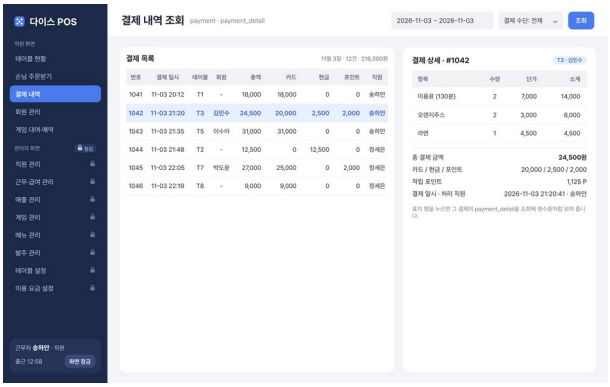
② 개발자 명 : 송하안 (학번 : 2025109056) — 회원 관리 · 결제 · 결제 내역 조회

담당 테이블 : member, payment, payment_detail / 참조 테이블 : table_usage, orders, order_detail, menu, staff

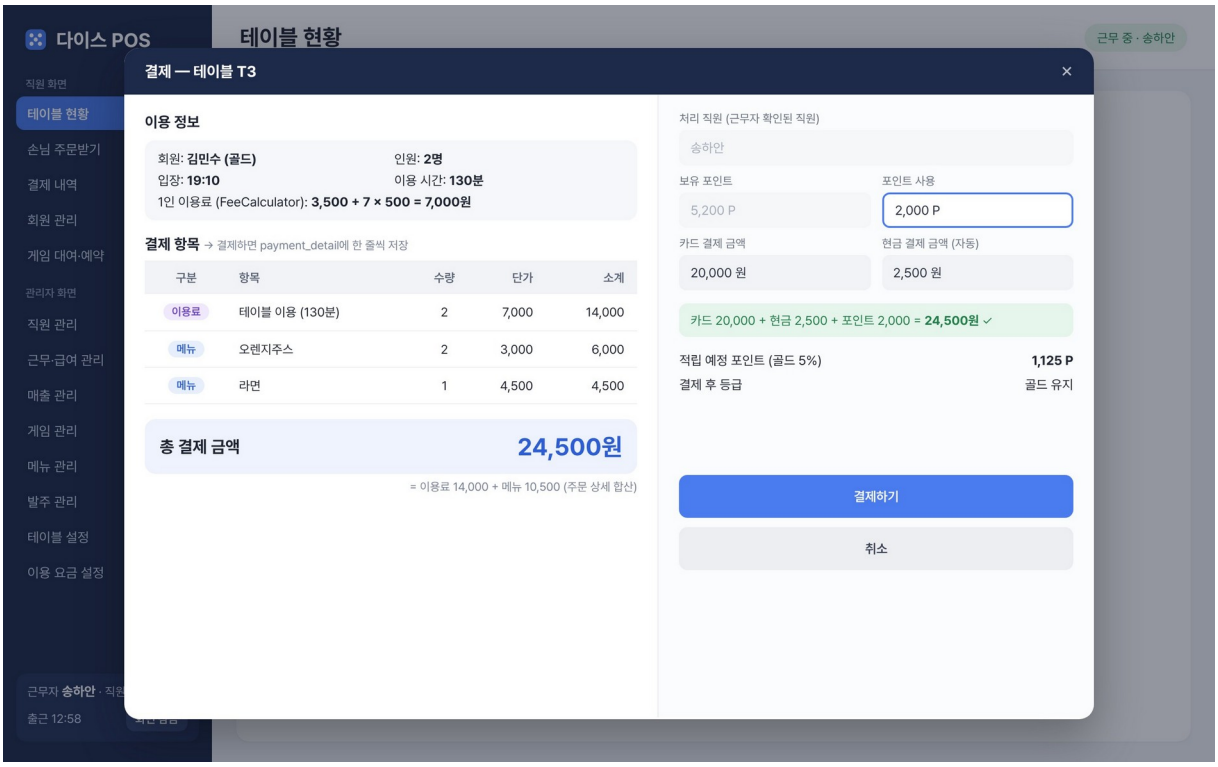
가. 개발할 화면 그림



[그림 ②-1] 회원 관리 [직원]



[그림 ②-2] 결제 내역 조회 [직원]



[그림 ②-3] 결제 팝업 [직원] — 결제 항목(이용료·메뉴)과 카드·현금·포인트 나눠 내기

나. 화면 기능 설명

화면	버튼 / 이벤트	동작 (DB 작업 포함)
회원 관리	화면 열림	member 를 SELECT 해서 회원 목록을 JTable 에 보여 준다. 누적 결제 금액은 payment 에서 member_id 별 SUM(total_amount)으로 계산해 표시한다.
회원 관리	검색창 입력 (KeyListener)	글자를 입력할 때마다 이름이나 전화번호 뒤 4 자리로 다시 조회한다.
회원 관리	표 행 클릭 (MouseListener)	고른 회원의 정보를 오른쪽 입력 폼에 채운다.

화면	버튼 / 이벤트	동작 (DB 작업 포함)
회원 관리	[등록] / [수정] / [삭제]	등록은 INSERT(등급 '일반', 포인트 0), 수정은 UPDATE, 삭제는 확인창 후 DELETE 한다. 결제 기록이 있는 회원은 삭제하지 않고 안내창을 띄운다. 등급은 직접 고치지 않고 결제할 때 자동으로 바뀐다.
회원 관리	입력 예외 처리	빈 칸, 전화번호 형식 오류, 전화번호 중복(UNIQUE 위반)이면 경고창을 띄운다.
결제	팝업 열림	table_usage 와 그 이용 건의 orders·order_detail 을 조회한다. 이용료는 ⑤의 FeeCalculator 로 1 인 요금을 구해 '이용료' 1 줄(수량 = 인원)로, 주문 상세는 같은 메뉴끼리 합쳐 '메뉴' 줄로 만들어 결제 항목 표와 총 결제 금액을 보여 준다.
결제	포인트 / 카드 금액 입력 (ChangeListener)	포인트는 보유 포인트와 총액을 넘지 못하게 막고, 카드 금액을 넣으면 현금 금액 = 총액 − 포인트 − 카드로 자동 계산한다. 카드 + 현금 + 포인트가 총액과 다르면 [결제하기]를 끈다.
결제	[결제하기] 버튼 (트랜잭션)	① payment 1 줄 INSERT ② 결제 항목마다 payment_detail INSERT ③ 회원 포인트 차감·적립, 누적 결제 금액으로 등급 갱신 ④ table_usage 를 '결제완료'로 변경 — 네 작업을 한 트랜잭션으로 처리하고 하나라도 실패하면 rollback 한다. 이미 결제된 이용 건이면(usage_id UNIQUE) 안내창을 띄운다.
결제 내역 조회	기간·결제 수단 선택 후 [조회]	기간 안의 payment 를 SELECT 해서 결제 번호, 일시, 테이블, 회원, 총액, 카드·현금·포인트, 처리 직원을 표에 보여 준다.
결제 내역 조회	표 행 클릭 (MouseListener)	고른 결제의 payment_detail 을 SELECT 해서 영수증처럼 항목·수량·단가·소계와 결제 수단별 금액, 적립 포인트를 보여 준다.

다. 연동 데이터베이스 구조

결제 정보는 **결제(payment)**와 **결제 상세(payment_detail)** 2 개 테이블로 저장한다. 결제 1 건에 대해 결제 테이블에는 총 결제 내역(일시, 직원, 회원, 총액, 카드·현금·포인트 결제 금액)이 1 줄 저장되고, 결제 상세 테이블에는 결제한 항목마다 1 줄씩 여러 줄이 저장된다.

테이블 : member (회원) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
member_id	INT (AUTO_INCREMENT)	PK	회원 번호
name	VARCHAR(20)		이름
phone1 ~ phone3	VARCHAR(3) / (4) / (4)	UNIQUE (3 개 묶음)	전화번호. 끝 4 자리로 회원 검색
gender	CHAR(1)		성별 (M / F)
grade	VARCHAR(10)		등급 (일반 / 실버 / 골드), 결제할 때 자동 변경
point	INT		보유 포인트, 기본값 0
join_date	DATE		가입일

테이블 : payment (결제) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
pay_id	INT (AUTO_INCREMENT)	PK	결제 번호
usage_id	INT	FK → table_usage.usage_id (UNIQUE)	결제한 테이블 이용 건 (한 이용 건은 한 번만 결제)
staff_id	INT	FK → staff.staff_id	결제를 처리한 직원

필드명	필드 자료형	PK / FK	필드 설명
member_id	INT	FK → member.member_id	결제한 회원 (비회원은 NULL)
pay_time	DATETIME		결제 일시 (연월일 시분초)
total_amount	INT		총 결제 금액 (= 결제 상세 소계의 합)
card_amount	INT		카드 결제 금액
cash_amount	INT		현금 결제 금액
point_amount	INT		포인트 결제 금액 (사용한 포인트)
earn_point	INT		이번 결제로 적립한 포인트 (비회원은 0)

테이블 : payment_detail (결제 상세) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
pay_detail_id	INT (AUTO_INCREMENT)	PK	결제 상세 번호
pay_id	INT	FK → payment.pay_id	어느 결제의 상세인지
item_type	VARCHAR(10)		항목 구분 (이용료 / 메뉴)
menu_id	INT	FK → menu.menu_id	상품(메뉴) 번호. 이용료 줄은 NULL
qty	INT		구매 수량 (이용료 줄은 이용 인원)
unit_price	INT		구매 단가 (메뉴는 menu.price 에서 불러옴, 이용료는 1 인 요금)
subtotal	INT		상품별 결제 금액 소계 (qty × unit_price)

[저장 예시] T3 테이블에서 김민수 회원(member_id 7) 2 명이 130 분 이용하고 **오렌지주스 2 잔과 라면 1 개**를 주문한 뒤, 포인트 2,000 점 + 카드 20,000 원 + 현금 2,500 원으로 결제한 경우 (1 인 이용료 = 3,500 + 7 × 500 = 7,000 원)

payment — 결제 1 건이므로 1 줄

필드	값	필드	값
pay_id	1042	**total_amount**	24500
usage_id	315 (T3 이용 건)	**card_amount**	20000
staff_id	4 (처리 직원)	**cash_amount**	2500
member_id	7 (김민수)	**point_amount**	2000
pay_time	2026-11-03 21:20:41	**earn_point**	1125 (카드·현금의 5%, 골드)

payment_detail — 결제한 항목이 3 개이므로 3 줄

pay_detail_id	pay_id	item_type	menu_id	qty	unit_price	subtotal
5120	1042	이용료	NULL	2 (인원)	7000	14000
5121	1042	메뉴	12 (오렌지주스)	2	3000	6000
5122	1042	메뉴	7 (라면)	1	4500	4500

- 결제한 **상품 번호와 개수**는 payment_detail 의 menu_id 와 qty 에 저장되고, 상품 이름은 menu 와 JOIN 해서 보여 준다.
- 단가는 결제할 때 menu.price 에서 불러와 unit_price 에 저장하므로, 나중에 메뉴 가격이 바뀌어도 지난 결제 금액은 그대로 남는다.

- 이용료는 메뉴가 아니므로 item_type 을 '이용료', menu_id 를 NULL 로 하고 수량에 인원, 단가에 1 인 요금을 넣는다.
- payment_detail 소계의 합(24,500) = payment.total_amount = card_amount + cash_amount + point_amount 가 되도록 결제 전에 검사한다.

테이블 : table_usage (테이블 이용) ☐ 담당 ☑ 참조 (담당: ⑤ 서윤채)

필드명	필드 자료형	PK / FK	필드 설명	사용
usage_id	INT (AUTO_INCREMENT)	PK	이용 번호	조회
table_no	INT	FK → cafe_table.table_no	이용한 테이블	조회
member_id	INT	FK → member.member_id	회원 번호 (비회원은 NULL)	조회
people	INT		이용 인원	조회
start_time	DATETIME		입장 시각	조회
end_time	DATETIME		퇴장 시각 (이용 중이면 NULL)	변경
status	VARCHAR(10)		상태 (이용중 / 결제완료)	조회·변경

테이블 : orders (주문) ☐ 담당 ☑ 참조 (담당: ⑥ 김하진)

필드명	필드 자료형	PK / FK	필드 설명	사용
order_id	INT (AUTO_INCREMENT)	PK	주문 번호	조회
usage_id	INT	FK → table_usage.usage_id	어느 테이블 이용 건의 주문인지 (후불제)	조회
staff_id	INT	FK → staff.staff_id	주문을 받은 직원 (키오스크 주문은 NULL)	
channel	VARCHAR(10)		주문 경로 (POS / 키오스크)	
order_time	DATETIME		주문 일시	
total_amount	INT		주문 금액 합계 (= 주문 상세 소계의 합)	
status	VARCHAR(10)		상태 (접수 / 확인). 키오스크 주문은 ' 접수'로 들어오고 직원이 확인하면 '확인'	

테이블 : order_detail (주문 상세) ☐ 담당 ☑ 참조 (담당: ⑥ 김하진)

필드명	필드 자료형	PK / FK	필드 설명	사용
order_detail_id	INT (AUTO_INCREMENT)	PK	주문 상세 번호	
order_id	INT	FK → orders.order_id	어느 주문의 상세인지	조회
menu_id	INT	FK → menu.menu_id	주문한 메뉴	조회
qty	INT		주문 수량	조회
unit_price	INT		주문 당시 단가 (menu.price 에서 불러옴)	조회
subtotal	INT		소계 (qty × unit_price)	조회

테이블 : menu (메뉴) ☐ 담당 ☑ 참조 (담당: ④ 박준영)

필드명	필드 자료형	PK / FK	필드 설명	사용
menu_id	INT (AUTO_INCREMENT)	PK	메뉴 번호	조회
name	VARCHAR(30)	UNIQUE	메뉴 이름 (중복 불가)	조회
category	VARCHAR(10)		분류 (음료 / 간식 / 식사)	
price	INT		판매 가격	
stock	INT		재고 수량, 기본값 0	
sold_out	TINYINT(1)		품절 여부 (1 이면 주문 불가)	

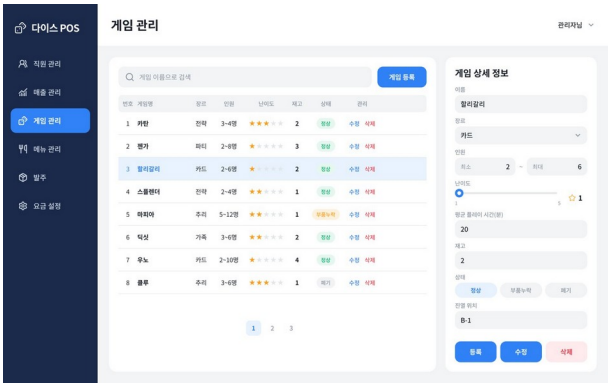
테이블 : staff (직원) ☐ 담당 ☑ 참조 (담당: ① 김재민)

필드명	필드 자료형	PK / FK	필드 설명	사용
staff_id	INT (AUTO_INCREMENT)	PK	직원 번호	조회
login_id	VARCHAR(20)	UNIQUE	아이디 (중복 불가)	
password	VARCHAR(64)		비밀번호 (SHA-256 으로 암호화한 값만 저장)	
name	VARCHAR(20)		이름	조회
phone1 ~ phone3	VARCHAR(3) / (4) / (4)		전화번호 (앞 / 가운데 / 끝자리)	
role	VARCHAR(10)		권한 (관리자 / 직원). 관리자만 관리자 화면을 열 수 있음	
emp_type	VARCHAR(10)		고용 형태 (정규직 / 계약직 / 시간제)	
hourly_wage	INT		시급 (시간제만 입력, 나머지는 NULL)	
monthly_pay	INT		월 기본급 (정규직·계약직만 입력)	
hire_date	DATE		입사일	
contract_end	DATE		계약 종료일 (계약직만 입력)	
status	VARCHAR(10)		재직 상태 (재직 / 퇴사)	
address	VARCHAR(100)		주소 (선택 입력)	
work_time	VARCHAR(20)		기본 근무 시간대 (예: 09:00~18:00, 선택 입력)	
memo	VARCHAR(500)		직원 메모 (선택 입력)	

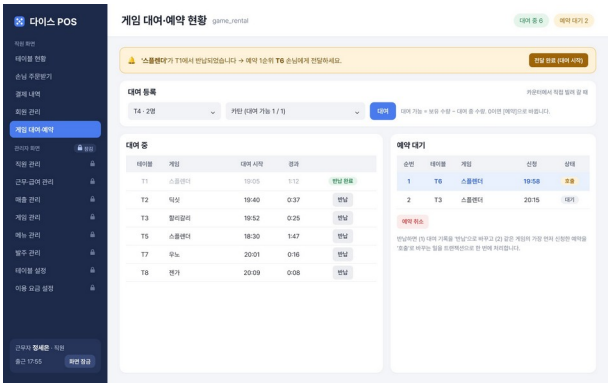
③ 개발자 명 : 김은별 (학번 : 2025104899) — 게임 관리 · 게임 대여·예약 · 키오스크(시작·게임 추천)

담당 테이블 : game, recommend, game_rental / 참조 테이블 : table_usage, cafe_table, member

가. 개발할 화면 그림



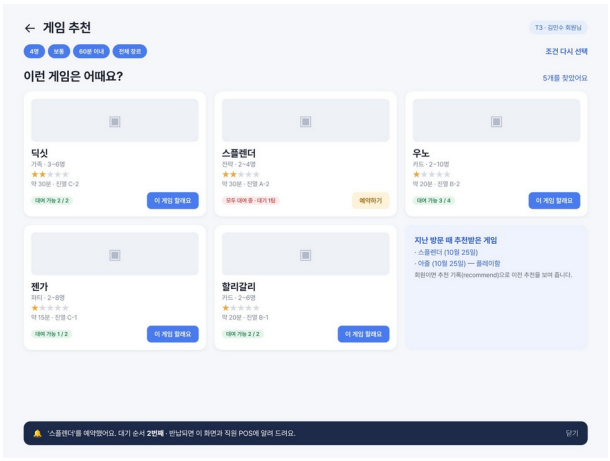
[그림 ③-1] 게임 관리 [관리자]



[그림 ③-2] 게임 대여·예약 현황 [직원]



[그림 ③-3] 키오스크 메인 (서비스 선택) [키오스크]



[그림 ③-4] 게임 추천 결과 · 대여 신청 · 예약 [키오스크]

나. 화면 기능 설명

화면	버튼 / 이벤트	동작 (DB 작업 포함)
게임 관리	화면 열림 / 이름 검색 (KeyListener)	game 을 SELECT 해서 표에 보여 주고, 검색창에 입력할 때마다 이름으로 다시 조회한다. recommend 를 game_id 별로 세어 '많이 추천된 게임 TOP 5'도 보여 준다.
게임 관리	표 행 클릭 / 난이도 슬라이더 (ChangeListener)	고른 게임의 정보를 오른쪽 폼에 채운다. 슬라이더를 움직이면 난이도만큼 ★을 바로 보여 준다.
게임 관리	[등록] / [수정] / [삭제]	새 게임은 보유 수량 0 으로 등록하고 ④ 발주에서 입고하면 수량이 늘어난다. 수정은 UPDATE, 삭제는 대여·추천 기록이 없는 게임만 확인창 후 DELETE 한다. 상태(정상 / 부품누락 / 폐기)는 ButtonGroup 으로 하나만 고른다.
게임 관리	[폐기 처리]	폐기할 수량을 입력받아 stock 을 줄이고, 0 이 되면 상태를 '폐기'로 바꾼다. 대여 중이 아닌 수량보다 많이 폐기하려 하면 경고창을 띄운다.
게임 관리	입력 예외 처리	빈 칸, 숫자가 아닌 값, 최소 인원이 최대 인원보다 크면 경고창을 띄운다.
게임 대여·예약	화면 열림 / 5 초마다 갱신 (Timer)	game_rental 에서 '대여중'과 '예약'인 기록을 SELECT 해서 대여 중 표와 예약 대기 표(신청 순서)를 보여 준다. 키오스크에서 들어온 신청도 바로 보인다.

화면	버튼 / 이벤트	동작 (DB 작업 포함)
게임 대여·예약	[대여] 버튼	대여 가능 수량(보유 수량 − 대여 중 수량)이 있으면 '대여중'으로, 없으면 '예약'으로 game_rental 에 INSERT 한다.
게임 대여·예약	[반납] → [전달 완료] (트랜잭션)	그 기록을 '반납'(return_time)으로 바꾸고, 같은 게임의 가장 먼저 신청한 '예약'을 찾아 호출 알림을 띄운다(한 트랜잭션). 직원이 게임을 건네고 [전달 완료]를 누르면 그 예약을 '대여중'(start_time)으로 바꾼다. [예약 취소]는 '취소'로 바꾼다.
키오스크 시작	설치할 때 테이블 지정	직원이 관리자 비밀번호를 넣고 이 키오스크의 테이블 번호를 고른다(cafe_table 에서 사용 중인 테이블 SELECT). 이후 손님은 테이블을 바꿀 수 없다.
키오스크 메인	서비스 카드 클릭 (CardLayout)	table_usage 에서 이 테이블이 이용 중인지 확인하고 게임 추천, 음식·음료 주문(⑥), 이용 현황(⑥) 화면으로 바꾼다. 이용 중이 아니면 "직원에게 입장 등록을 요청해 주세요"를 띄운다.
게임 추천	조건 선택 후 [추천받기]	인원·난이도·시간·장르 조건에 맞고 상태가 '정상'인 게임을 SELECT 해서 대여 가능 수량과 함께 카드로 보여 주고, 추천한 게임마다 recommend 에 INSERT(이용 건, 회원, 게임) 한다. 회원이면 지난 방문 때 추천받은 게임도 보여 준다. 결과가 없으면 안내 문구를 띄운다.
게임 추천	[이 게임 할래요] / [예약하기]	대여 가능하면 game_rental 에 '대여중'으로 INSERT 하고 진열 위치를 안내한다. 모두 대여 중이면 '예약'으로 INSERT 하고 대기 순서를 알려 준다. 두 경우 모두 recommend.chosen 을 1 로 바꾸고, 같은 게임을 이미 예약했으면 중복 예약을 막는다.

다. 연동 데이터베이스 구조

테이블 : game (보드게임) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
game_id	INT (AUTO_INCREMENT)	PK	게임 번호
name	VARCHAR(50)		게임 이름
genre	VARCHAR(10)		장르 (파티 / 가족 / 전략 / 추리 / 카드)
min_player	INT		최소 인원
max_player	INT		최대 인원
difficulty	INT		난이도 1~5 (★로 표시)
play_min	INT		평균 플레이 시간(분)
stock	INT		보유 수량, 기본값 0 (입고하면 증가, 폐기하면 감소)
status	VARCHAR(10)		상태 (정상 / 부품누락 / 폐기)
location	VARCHAR(10)		진열 위치 (예: A-1)

테이블 : recommend (게임 추천 기록) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
rec_id	INT (AUTO_INCREMENT)	PK	추천 번호
usage_id	INT	FK → table_usage.usage_id	추천을 받은 테이블 이용 건

필드명	필드 자료형	PK / FK	필드 설명
member_id	INT	FK → member.member_id	추천받은 회원 (비회원은 NULL)
game_id	INT	FK → game.game_id	추천한 게임
rec_time	DATETIME		추천한 시각
chosen	TINYINT(1)		손님이 이 게임을 골랐는지 (대여 신청·예약하면 1)

테이블 : game_rental (게임 대여·예약) ☑️ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
rental_id	INT (AUTO_INCREMENT)	PK	대여 번호
game_id	INT	FK → game.game_id	대여·예약한 게임
usage_id	INT	FK → table_usage.usage_id	어느 테이블 이용 건인지
status	VARCHAR(10)		상태 (예약 / 대여중 / 반납 / 취소)
request_time	DATETIME		신청 시각 (예약 대기 순서의 기준)
start_time	DATETIME		대여 시작 시각 (게임을 받은 시각)
return_time	DATETIME		반납 시각 (대여 중이면 NULL)

- **추천 기록(recommend)** : 손님과 게임의 매칭 정보를 추천할 때마다 1 줄씩 저장한다. 회원이면 member_id 가 들어가 다음 방문 때 지난 추천을 보여 줄 수 있고, 비회원도 usage_id 로 어느 테이블에 추천했는지 남는다. chosen 으로 추천한 게임을 실제로 골랐는지 알 수 있다.
- **대여·예약(game_rental)** : 예약은 '아직 시작하지 않은 대여'이므로 한 테이블에 상태(예약 → 대여중 → 반납, 또는 취소)로 관리한다. 대기 순서는 같은 게임의 '예약' 중 request_time 순서이다.

테이블 : table_usage (테이블 이용) ☐ 담당 ☑️ 참조 (담당: ⓢ 서윤채)

필드명	필드 자료형	PK / FK	필드 설명	사용
usage_id	INT (AUTO_INCREMENT)	PK	이용 번호	조회
table_no	INT	FK → cafe_table.table_no	이용한 테이블	조회
member_id	INT	FK → member.member_id	회원 번호 (비회원은 NULL)	조회
people	INT		이용 인원	
start_time	DATETIME		입장 시각	
end_time	DATETIME		퇴장 시각 (이용 중이면 NULL)	
status	VARCHAR(10)		상태 (이용중 / 결제완료)	조회

테이블 : cafe_table (카페 테이블) ☐ 담당 ☑️ 참조 (담당: ⓢ 서윤채)

필드명	필드 자료형	PK / FK	필드 설명	사용
table_no	INT	PK	테이블 번호	조회
table_name	VARCHAR(10)	UNIQUE	표시 이름 (예: T1, 룸 A)	조회

필드명	필드 자료형	PK / FK	필드 설명	사용
capacity	INT		정원 (처음 등록·공사 시에만 수정, 인원 초과 검사)	
is_active	TINYINT(1)		사용 여부 (1 사용가능 / 0 사용불가: 수리 등)	조회

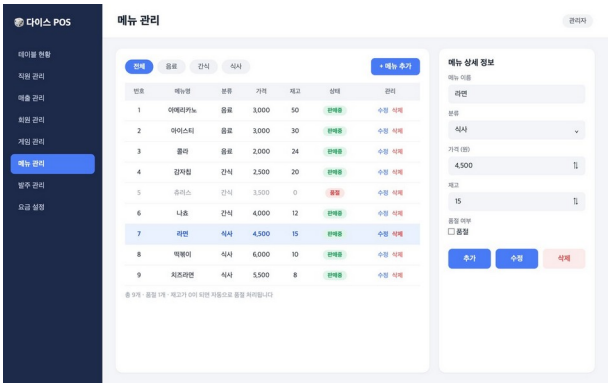
테이블 : member (회원) ☐ **담당** ☑️ **참조** (담당: ② 송하안)

필드명	필드 자료형	PK / FK	필드 설명	사용
member_id	INT (AUTO_INCREMENT)	PK	회원 번호	조회
name	VARCHAR(20)		이름	조회
phone1 ~ phone3	VARCHAR(3) / (4) / (4)	UNIQUE (3 개 묶음)	전화번호. 끝 4 자리로 회원 검색	
gender	CHAR(1)		성별 (M / F)	
grade	VARCHAR(10)		등급 (일반 / 실버 / 골드), 결제할 때 자동 변경	
point	INT		보유 포인트, 기본값 0	
join_date	DATE		가입일	

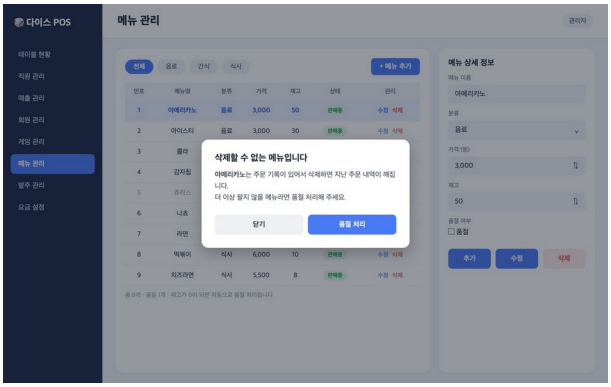
④ 개발자 명 : 박준영 (학번 : 2024081106) — 메뉴 관리 · 발주 관리

담당 테이블 : menu, purchase_order, purchase_order_detail / 참조 테이블 : game, staff, order_detail

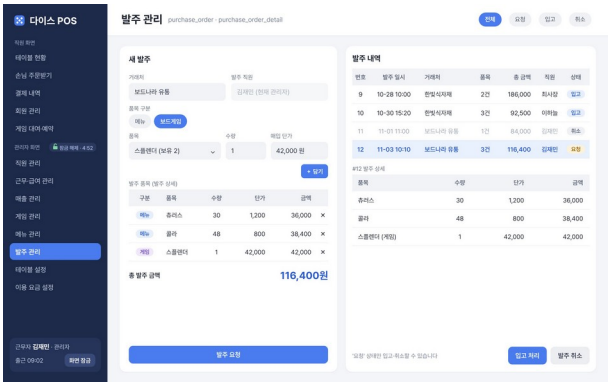
가. 개발할 화면 그림



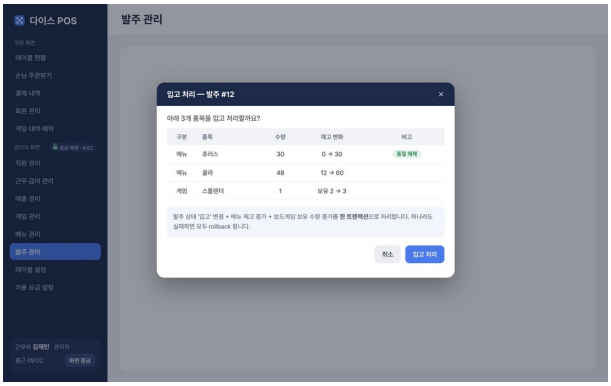
[그림 ④-1] 메뉴 관리 [관리자]



[그림 ④-2] 주문 기록이 있는 메뉴 삭제 안내 [관리자]



[그림 ④-3] 발주 관리 (메뉴·보드게임) [관리자]



[그림 ④-4] 입고 처리 확인 팝업 [관리자]

나. 화면 기능 설명

화면	버튼 / 이벤트	동작 (DB 작업 포함)
메뉴 관리	화면 열림 / 분류 버튼	menu 를 SELECT 해서 표에 보여 준다. 음료·간식·식사 버튼을 누르면 분류별로 다시 조회한다. 품절 메뉴는 회색으로 표시한다.
메뉴 관리	표 행 클릭 (MouseListener)	고른 메뉴의 정보를 오른쪽 폼에 채운다.
메뉴 관리	[추가] / [수정]	입력값을 검사한 뒤 menu 에 INSERT 또는 UPDATE 하고 표를 새로 고친다. 이름이 비었거나 중복이거나 가격이 0 이하면 경고창을 띄운다.
메뉴 관리	품절 체크박스 (ItemListener)	체크하면 sold_out 을 1 로 바꾼다. 품절 메뉴는 ⑥ 주문 화면에서 고를 수 없다. ⑥ 에서 주문해 재고가 0 이 되거나 재고를 0 으로 저장해도 자동으로 품절이 된다.
메뉴 관리	[삭제] 버튼	order_detail 에 그 메뉴의 주문 기록이 있는지 먼저 확인한다(SELECT COUNT). 기록이 없으면 DELETE 하고, 있으면 삭제 대신 품절 처리하라는 안내창을 띄운다.
발주 관리	품목 구분 선택 (ItemListener)	'메뉴'를 고르면 menu 목록, '보드게임'을 고르면 game 목록을 품목 콤보박스에 채운다. 품목을 고르면 같은 품목의 직전 발주 단가를 불러와 매입 단가 칸에 넣는다(바꿀 수 있음).
발주 관리	[담기] 버튼	품목·수량·단가를 발주 품목 표에 한 줄 추가하고 금액(수량 × 단가)과 총 발주 금액을 다시 계산한다. 같은 품목은 수량만 합친다. 수량·단가가 0 이하면 경고창을 띄운다.

화면	버튼 / 이벤트	동작 (DB 작업 포함)
발주 관리	[발주 요청] 버튼 (트랜잭션)	purchase_order 1 줄(상태 '요청', 총 발주 금액, 발주 직원 = 확인된 관리자)과 품목마다 purchase_order_detail 을 INSERT 한다. 하나라도 실패하면 rollback 한다.
발주 관리	발주 내역 행 선택 (ListSelectionListener) / [발주 취소]	고른 발주의 상세를 아래 표에 보여 주고, '요청' 상태일 때만 [입고 처리]·[발주 취소] 버튼을 켠다. 취소는 확인창 후 상태를 '취소'로 UPDATE 해서 기록을 남긴다.
발주 관리	[입고 처리] 버튼 (트랜잭션)	발주 상태를 '입고'(receive_date)로 바꾸고, 상세마다 메뉴면 menu.stock 을, 보드게임이면 game.stock 을 수량만큼 늘린다(메뉴 재고가 0 이었으면 품절 해제). 모두 한 트랜잭션으로 묶어 하나라도 실패하면 rollback 한다.

다. 연동 데이터베이스 구조

테이블 : menu (메뉴) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
menu_id	INT (AUTO_INCREMENT)	PK	메뉴 번호
name	VARCHAR(30)	UNIQUE	메뉴 이름 (중복 불가)
category	VARCHAR(10)		분류 (음료 / 간식 / 식사)
price	INT		판매 가격
stock	INT		재고 수량, 기본값 0
sold_out	TINYINT(1)		품절 여부 (1 이면 주문 불가)

테이블 : purchase_order (발주) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
po_id	INT (AUTO_INCREMENT)	PK	발주 번호
staff_id	INT	FK → staff.staff_id	발주한 직원
supplier	VARCHAR(30)		거래처 (선택 입력)
order_date	DATETIME		발주 일시
total_amount	INT		총 발주 금액 (= 발주 상세 금액의 합)
status	VARCHAR(10)		상태 (요청 / 입고 / 취소)
receive_date	DATETIME		입고 일시 (입고 전이면 NULL)

테이블 : purchase_order_detail (발주 상세) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
po_detail_id	INT (AUTO_INCREMENT)	PK	발주 상세 번호
po_id	INT	FK → purchase_order.po_id	어느 발주의 상세인지
item_type	VARCHAR(10)		품목 구분 (메뉴 / 보드게임)
menu_id	INT	FK → menu.menu_id	발주한 메뉴 (보드게임이면 NULL)
game_id	INT	FK → game.game_id	발주한 보드게임 (메뉴면 NULL)

필드명	필드 자료형	PK / FK	필드 설명
qty	INT		발주 수량
unit_cost	INT		매입 단가 (같은 품목의 직전 발주 단가를 불러오고 수정 가능)
amount	INT		발주 금액 (qty × unit_cost)

- **보드게임 발주** : 발주 상세의 item_type 으로 메뉴와 보드게임을 구분하고, 메뉴면 menu_id, 보드게임이면 game_id 를 채운다. 기존 게임 추가 구입은 바로 발주하고, 새 게임은 ③ 게임 관리에서 보유 수량 0 으로 먼저 등록한 뒤 발주한다. 폐기는 ③ 게임 관리의 [폐기 처리]에서 한다.
- **단가와 발주 금액** : 품목마다 매입 단가(unit_cost)와 발주 금액(amount = 수량 × 단가)을 저장하고, 발주 1 건의 합계는 purchase_order.total_amount 에 저장한다.
- **주문 테이블을 참조하는 이유** : 주문 기록이 있는 메뉴를 지우면 지난 주문·결제 내역에서 메뉴를 찾을 수 없게 되므로, 삭제 전에 주문 기록이 있는지 확인하는 데만 쓴다. 주문이 orders + order_detail 로 나뉘어 메뉴 번호는 order_detail 에 있으므로 order_detail 을 참조한다.

테이블 : game (보드게임) ☐ **담당** ☑ **참조** (담당: ③ 김은별)

필드명	필드 자료형	PK / FK	필드 설명	사용
game_id	INT (AUTO_INCREMENT)	PK	게임 번호	**조회**
name	VARCHAR(50)		게임 이름	**조회**
genre	VARCHAR(10)		장르 (파티 / 가족 / 전략 / 추리 / 카드)	
min_player	INT		최소 인원	
max_player	INT		최대 인원	
difficulty	INT		난이도 1~5 (★로 표시)	
play_min	INT		평균 플레이 시간(분)	
stock	INT		보유 수량, 기본값 0 (입고하면 증가, 폐기하면 감소)	**변경**
status	VARCHAR(10)		상태 (정상 / 부품누락 / 폐기)	
location	VARCHAR(10)		진열 위치 (예: A-1)	

테이블 : staff (직원) ☐ **담당** ☑ **참조** (담당: ① 김재민)

필드명	필드 자료형	PK / FK	필드 설명	사용
staff_id	INT (AUTO_INCREMENT)	PK	직원 번호	**조회**
login_id	VARCHAR(20)	UNIQUE	아이디 (중복 불가)	
password	VARCHAR(64)		비밀번호 (SHA-256 으로 암호화한 값만 저장)	
name	VARCHAR(20)		이름	**조회**
phone1 ~ phone3	VARCHAR(3) / (4) / (4)		전화번호 (앞 / 가운데 / 끝자리)	
role	VARCHAR(10)		권한 (관리자 / 직원). 관리자만 관리자 화면을 열 수 있음	
emp_type	VARCHAR(10)		고용 형태 (정규직 / 계약직 / 시간제)	

필드명	필드 자료형	PK / FK	필드 설명	사용
hourly_wage	INT		시급 (시간제만 입력, 나머지는 NULL)	
monthly_pay	INT		월 기본급 (정규직·계약직만 입력)	
hire_date	DATE		입사일	
contract_end	DATE		계약 종료일 (계약직만 입력)	
status	VARCHAR(10)		재직 상태 (재직 / 퇴사)	
address	VARCHAR(100)		주소 (선택 입력)	
work_time	VARCHAR(20)		기본 근무 시간대 (예: 09:00~18:00, 선택 입력)	
memo	VARCHAR(500)		직원 메모 (선택 입력)	

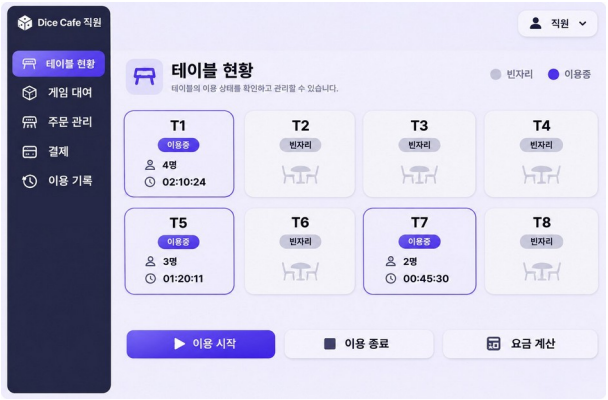
테이블 : order_detail (주문 상세) ☐ **담당** ☑️ **참조** (담당: ⑥ 김하진)

필드명	필드 자료형	PK / FK	필드 설명	사용
order_detail_id	INT (AUTO_INCREMENT)	PK	주문 상세 번호	
order_id	INT	FK → orders.order_id	어느 주문의 상세인지	
menu_id	INT	FK → menu.menu_id	주문한 메뉴	**조회**
qty	INT		주문 수량	
unit_price	INT		주문 당시 단가 (menu.price 에서 불러옴)	
subtotal	INT		소계 (qty × unit_price)	

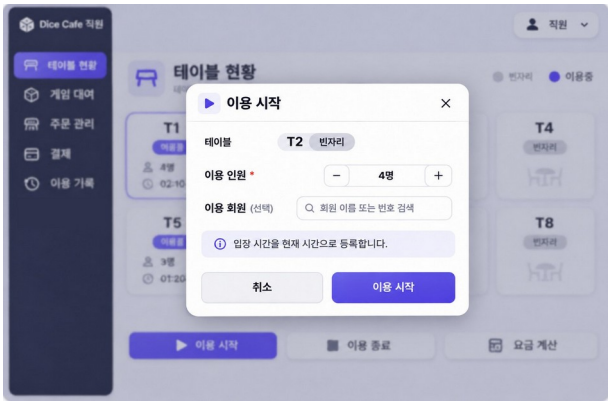
⑤ 개발자 명 : 서윤채 (학번 : 2025101273) — 테이블 현황 · 테이블 설정 · 이용 요금 설정

담당 테이블 : cafe_table, table_usage, fee_setting / 참조 테이블 : member, game_rental

가. 개발할 화면 그림



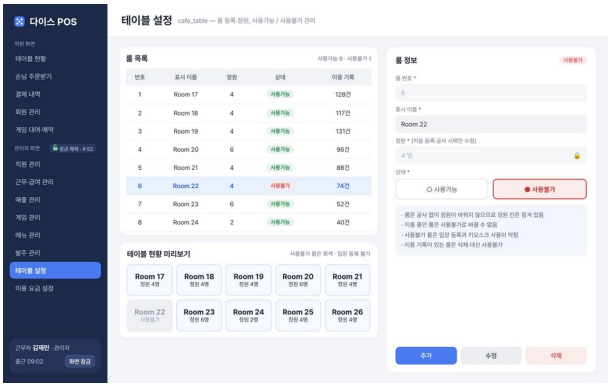
[그림 ⑤-1] 테이블 현황 [직원]



[그림 ⑤-2] 이용 시작 팝업 [직원]



[그림 ⑤-3] 이용 종료 팝업 [직원]



[그림 ⑤-4] 테이블 설정 [관리자]



[그림 ⑤-5] 이용 요금 설정 [관리자]

나. 화면 기능 설명

화면	버튼 / 이벤트	동작 (DB 작업 포함)
테이블 현황	화면 열림 / 1 분마다 갱신 (Thread)	cafe_table 을 읽어 카드를 만들고(개수에 맞춰 GridLayout 행 수 계산), 사용불가 룸은 회색으로 표시해 고를 수 없게 한다. table_usage 로 이용 중인 카드에 인원과 경과 시간을 표시하고, 스레드로 1 분마다 갱신한다.
테이블 현황	카드 클릭 (MouseListener)	고른 테이블을 강조한다. 빈자리면 [이용 시작], 이용 중이면 [이용 종료]·[요금 계산]을 쓸 수 있다.

화면	버튼 / 이벤트	동작 (DB 작업 포함)
테이블 현황	[이용 시작] 팝업	인원과 회원(이름·전화번호 뒤 4 자리로 member 검색, 선택)을 입력하면 table_usage 에 INSERT 한다(상태 '이용중', 입장 시각은 현재 시각). 인원이 좌석 수보다 많으면 경고창을 띄운다.
테이블 현황	[이용 종료] 팝업	이용 시간과 요금을 보여 준다. 반납하지 않은 게임(game_rental '대여중')이 있으면 먼저 반납하라고 안내한다. [결제하기]를 누르면 ②의 결제 팝업을 열고, 결제가 끝나면 카드가 빈자리로 바뀐다.
테이블 현황	[요금 계산] 버튼	FeeCalculator 로 지금까지의 요금만 계산해서 보여 준다. DB 는 바꾸지 않는다.
테이블 설정	화면 열림 / 목록 클릭	cafe_table 을 SELECT 해서 번호, 표시 이름, 좌석 수, 사용 여부, 이용 기록 수를 보여 주고, 고른 행을 오른쪽 폼에 채운다. 아래에 테이블 현황 미리보기를 보여 준다.
테이블 설정	[추가] / [수정]	번호·이름 중복, 정원(1 이상)을 검사한 뒤 INSERT·UPDATE 한다. 정원은 처음 등록하거나 공사로 룸 구조가 바뀔 때만 수정한다.
테이블 설정	사용 여부 체크박스 (ItemListener)	체크를 풀면 is_active = 0(사용불가: 고장·수리 등), 다시 체크하면 1(사용가능)로 UPDATE 한다. 이용 중인 룸은 사용불가로 바꿀 수 없다. 사용불가 룸은 입장 등록과 키오스크 사용이 막힌다.
테이블 설정	[삭제]	이용 기록이 없는 룸만 확인창 후 DELETE 한다. 기록이 있으면 삭제 대신 사용불가로 바꾸라고 안내한다.
이용 요금 설정	화면 열림 / 목록 클릭	fee_setting 을 SELECT 해서 요금 목록을 보여 주고, 고른 행을 왼쪽 입력 폼에 채운다.
이용 요금 설정	[등록] / [수정] / [삭제]	기본 시간, 1 인 기본요금, 추가 단위, 1 인 추가요금, 적용 시작일을 입력해 INSERT·UPDATE 하고, 삭제는 확인창 후 DELETE 한다. 이미 적용이 시작된 요금은 지난 결제에 쓰였으므로 고치지 않고 새 요금을 등록하게 한다.
공용 클래스	FeeCalculator	적용 시작일이 오늘 이전인 요금 중 가장 최근 것으로 1 인 요금 = 기본요금 + 올림(초과 시간 ÷ 추가 단위) × 추가요금을 계산하고, 인원을 곱해 이용료를 구한다. ② 결제와 ⑥ 이용 현황에서도 같이 쓴다. (예: 130 분 → 3,500 + 7 × 500 = 1 인 7,000 원)

다. 연동 데이터베이스 구조

테이블 설정에 대한 부연 설명 : 우리 카페는 룸형 구조라서 룸마다 정원이 정해져 있고, 정원은 공사 없이는 바뀌지 않는다. 그래서 테이블 설정 화면의 정원 입력·수정은 처음 등록하거나 공사로 구조가 바뀔 때만 쓴다. 대신 고장이나 수리로 룸을 잠시 쓸 수 없을 때 손님이 배정되지 않도록, 관리자가 룸을 **사용가능 / 사용불가**로 바꿀 수 있게 한다. 사용불가인 룸은 테이블 현황에 회색으로 표시되고 입장 등록과 키오스크 사용이 막힌다. (매장 구성은 2.2 (5) 참고 사진)

테이블 : cafe_table (카페 테이블) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
table_no	INT	PK	테이블 번호
table_name	VARCHAR(10)	UNIQUE	표시 이름 (예: T1, 룸 A)
capacity	INT		정원 (처음 등록·공사 시에만 수정, 인원 초과 검사)
is_active	TINYINT(1)		사용 여부 (1 사용가능 / 0 사용불가: 수리 등)

테이블 : table_usage (테이블 이용) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
usage_id	INT (AUTO_INCREMENT)	PK	이용 번호
table_no	INT	FK → cafe_table.table_no	이용한 테이블

필드명	필드 자료형	PK / FK	필드 설명
member_id	INT	FK → member.member_id	회원 번호 (비회원은 NULL)
people	INT		이용 인원
start_time	DATETIME		입장 시각
end_time	DATETIME		퇴장 시각 (이용 중이면 NULL)
status	VARCHAR(10)		상태 (이용중 / 결제완료)

테이블 : fee_setting (이용 요금 설정) ☑ 담당 ☐ 참조

필드명	필드 자료형	PK / FK	필드 설명
fee_id	INT (AUTO_INCREMENT)	PK	요금 번호
base_minutes	INT		기본 시간(분)
base_price	INT		1 인 기본요금
extra_unit	INT		추가 요금 단위(분)
extra_price	INT		1 인 추가요금
apply_date	DATE		적용 시작일

테이블 : member (회원) ☐ 담당 ☑ 참조 (담당: ② 송하안)

필드명	필드 자료형	PK / FK	필드 설명	사용
member_id	INT (AUTO_INCREMENT)	PK	회원 번호	조회
name	VARCHAR(20)		이름	조회
phone1 ~ phone3	VARCHAR(3) / (4) / (4)	UNIQUE (3 개 묶음)	전화번호. 끝 4 자리로 회원 검색	조회
gender	CHAR(1)		성별 (M / F)	
grade	VARCHAR(10)		등급 (일반 / 실버 / 골드), 결제할 때 자동 변경	
point	INT		보유 포인트, 기본값 0	
join_date	DATE		가입일	

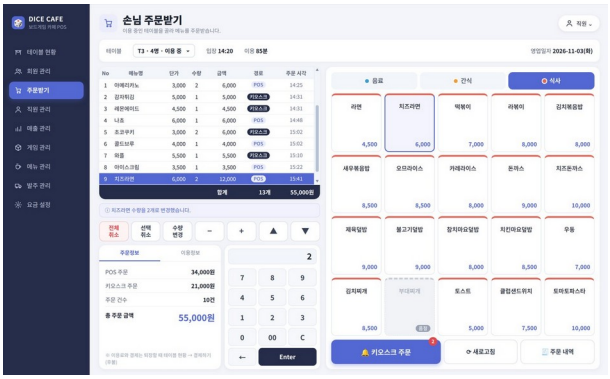
테이블 : game_rental (게임 대여·예약) ☐ 담당 ☑ 참조 (담당: ③ 김은별)

필드명	필드 자료형	PK / FK	필드 설명	사용
rental_id	INT (AUTO_INCREMENT)	PK	대여 번호	
game_id	INT	FK → game.game_id	대여·예약한 게임	
usage_id	INT	FK → table_usage.usage_id	어느 테이블 이용 건인지	조회
status	VARCHAR(10)		상태 (예약 / 대여중 / 반납 / 취소)	조회
request_time	DATETIME		신청 시각 (예약 대기 순서의 기준)	
start_time	DATETIME		대여 시작 시각 (게임을 받은 시각)	
return_time	DATETIME		반납 시각 (대여 중이면 NULL)	

⑥ 개발자 명 : 김하진 (학번 : 2025105269) — 손님 주문받기(POS) · 키오스크 주문 · 이용 현황

담당 테이블 : orders, order_detail / 참조 테이블 : menu, table_usage

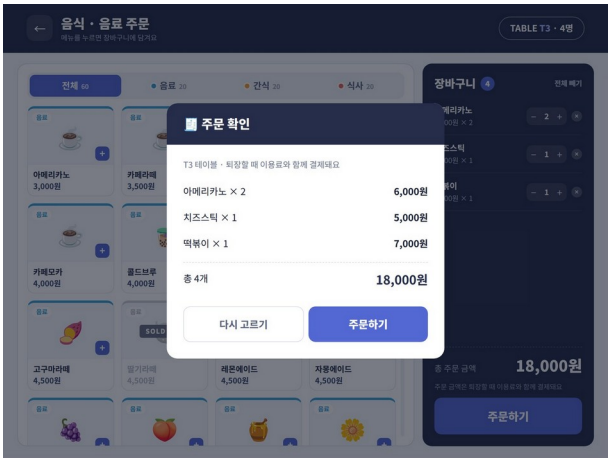
가. 개발할 화면 그림



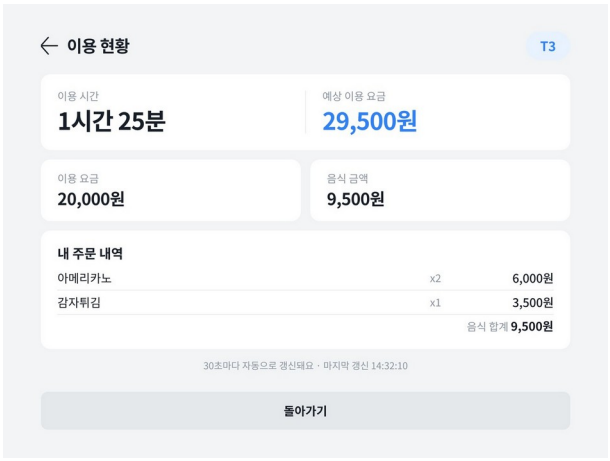
[그림 ⑥-1] 손님 주문받기 [직원]



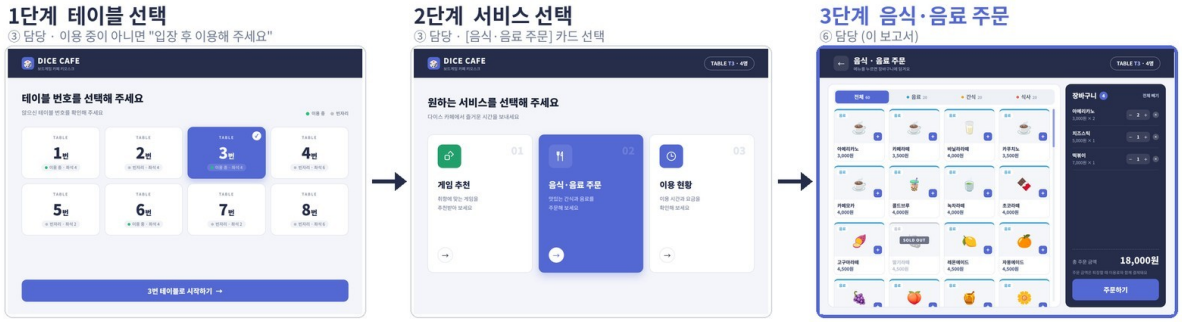
[그림 ⑥-2] 키오스크 주문 · 장바구니 [키오스크]



[그림 ⑥-3] 키오스크 주문 확인 팝업 [키오스크]



[그림 ⑥-4] 이용 현황 [키오스크] (③ 에서 옮겨 옴)



[그림 ⑥-5] 키오스크 진입 흐름 (1 단계 테이블 지정은 설치할 때 직원이 한 번만 설정, 2 단계는 ③, 3 단계부터 ⑥)

나. 화면 기능 설명

화면	버튼 / 이벤트	동작 (DB 작업 포함)
손님 주문받기	테이블 선택 (JComboBox, ItemListener)	이용 중인 테이블만 콤보박스에 넣고, 고르면 그 이용 건의 orders 와 order_detail 을 JOIN 해서 주문 내역(메뉴, 수량, 금액, 주문 경로)과 합계를 표에 보여 준다.

화면	버튼 / 이벤트	동작 (DB 작업 포함)
손님 주문받기	분류 버튼 / 메뉴 버튼 클릭	분류별로 menu 를 조회해 메뉴 버튼을 만든다. 메뉴 버튼을 누르면 주문 대기 목록에 담고(같은 메뉴는 수량 +1), 품절 메뉴 버튼은 비활성화한다.
손님 주문받기	[주문] 버튼 (트랜잭션)	orders 1 줄(경로 'POS', 상태 '확인', 받은 직원)과 메뉴마다 order_detail(단가는 menu.price 에서 불러옴)을 INSERT 하고 메뉴 재고를 줄인다(④ DAO, 0 이면 품절). 하나라도 실패하면 rollback 한다.
손님 주문받기	+ / − , 수량 변경 (키패드)	고른 order_detail 의 qty 와 subtotal 을 UPDATE 하고 orders.total_amount 를 다시 계산한다. 0 이나 빈 값, 재고보다 큰 수면 경고창을 띄운다.
손님 주문받기	[선택 취소] / [전체 취소]	확인창 후 고른 주문 상세 또는 그 테이블의 주문 전체를 DELETE 한다. 상세를 먼저 지우고, 상세가 하나도 없어진 주문(헤더)도 지운다. 취소한 수량만큼 재고를 되돌린다.
손님 주문받기	키오스크 주문 알림 (Timer, 5 초)	5 초마다 경로 '키오스크', 상태 '접수'인 orders 수를 조회해서 버튼에 숫자로 표시하고, 직원이 확인하면 '확인'으로 UPDATE 한다.
키오스크 주문	분류 탭 / 메뉴 카드 클릭	분류별 메뉴 카드를 보여 주고, 카드를 누르면 장바구니(ArrayList, JList)에 담는다. 같은 메뉴는 수량만 늘린다. DB 는 바꾸지 않는다.
키오스크 주문	장바구니 − / + / ✕	장바구니 안에서 수량을 바꾸거나 빼고 총액을 다시 계산한다. 비어 있으면 [주문하기] 버튼을 끈다.
키오스크 주문	[주문하기] → 확인 팝업 (트랜잭션)	장바구니 전체를 orders 1 줄(경로 '키오스크', 상태 '접수') + 메뉴마다 order_detail 1 줄로 한 번에 저장하고 재고를 줄인다. 실패하면 rollback 한다.
이용 현황	화면 열림 / 30 초마다 갱신 (Thread)	이 테이블의 table_usage 에서 입장 시각과 인원을 읽어 이용 시간을 계산하고, 이용 요금(⑤ 의 FeeCalculator), 주문 내역(order_detail), 음식 금액, 예상 요금을 보여 준다. 스레드로 30 초마다 다시 계산한다.

다. 연동 데이터베이스 구조

한 번의 주문에 메뉴가 여러 개일 수 있으므로, 주문도 결제와 같은 방식으로 **주문(orders)**과 **주문 상세(order_detail)** 2 개 테이블로 저장한다. 주문 1 번에 orders 1 줄, 주문한 메뉴마다 order_detail 1 줄이 저장된다.

테이블 : orders (주문) ☑️ **담당** ☐ **참조**

필드명	필드 자료형	PK / FK	필드 설명
order_id	INT (AUTO_INCREMENT)	PK	주문 번호
usage_id	INT	FK → table_usage.usage_id	어느 테이블 이용 건의 주문인지 (후불제)
staff_id	INT	FK → staff.staff_id	주문을 받은 직원 (키오스크 주문은 NULL)
channel	VARCHAR(10)		주문 경로 (POS / 키오스크)
order_time	DATETIME		주문 일시
total_amount	INT		주문 금액 합계 (= 주문 상세 소계의 합)
status	VARCHAR(10)		상태 (접수 / 확인). 키오스크 주문은 '접수'로 들어오고 직원이 확인하면 '확인'

테이블 : order_detail (주문 상세) ☑️ **담당** ☐ **참조**

필드명	필드 자료형	PK / FK	필드 설명
order_detail_id	INT (AUTO_INCREMENT)	PK	주문 상세 번호
order_id	INT	FK → orders.order_id	어느 주문의 상세인지

필드명	필드 자료형	PK / FK	필드 설명
menu_id	INT	FK → menu.menu_id	주문한 메뉴
qty	INT		주문 수량
unit_price	INT		주문 당시 단가 (menu.price 에서 불러옴)
subtotal	INT		소계 (qty × unit_price)

[저장 예시] T3 손님이 키오스크에서 오렌지주스 2 잔과 라면 1 개를 한 번에 주문한 경우 — orders 1 줄 + order_detail 2 줄

order_id	usage_id	staff_id	channel	order_time	total_amount	status
881	315	NULL	키오스크	2026-11-03 19:42:10	10500	접수

order_detail_id	order_id	menu_id	qty	unit_price	subtotal
2301	881	12 (오렌지주스)	2	3000	6000
2302	881	7 (라면)	1	4500	4500

테이블 : menu (메뉴) ☐ **담당** ☑️ **참조** (담당: ④ 박준영)

필드명	필드 자료형	PK / FK	필드 설명	사용
menu_id	INT (AUTO_INCREMENT)	PK	메뉴 번호	조회
name	VARCHAR(30)	UNIQUE	메뉴 이름 (중복 불가)	조회
category	VARCHAR(10)		분류 (음료 / 간식 / 식사)	조회
price	INT		판매 가격	조회
stock	INT		재고 수량, 기본값 0	조회·변경
sold_out	TINYINT(1)		품절 여부 (1 이면 주문 불가)	조회·변경

테이블 : table_usage (테이블 이용) ☐ **담당** ☑️ **참조** (담당: ⑤ 서윤채)

필드명	필드 자료형	PK / FK	필드 설명	사용
usage_id	INT (AUTO_INCREMENT)	PK	이용 번호	조회
table_no	INT	FK → cafe_table.table_no	이용한 테이블	조회
member_id	INT	FK → member.member_id	회원 번호 (비회원은 NULL)	
people	INT		이용 인원	조회
start_time	DATETIME		입장 시각	조회
end_time	DATETIME		퇴장 시각 (이용 중이면 NULL)	
status	VARCHAR(10)		상태 (이용중 / 결제완료)	조회

4. 벤치마킹 프로그램

OKPOS (카페·음식점 POS)

조사 내용 : 후불 매장에서는 테이블 화면에서 테이블을 고른 뒤, 주문 화면에서 메뉴를 누르고 [주문]을 눌러 주문을 넣는다. 이미 주문한 메뉴는 "선택 취소"(고른 메뉴만)와 "전체 취소"(테이블 주문 전체)로 나눠서 취소할 수 있다. 수량은 메뉴를 고르고 숫자를 입력해서 바꾼다. 주문할 때 고객 수와 요청사항을 같이 적을 수 있다. 손님이 자리를 옮기면 테이블 이동·합석 기능으로 주문을 다른 테이블로 옮길 수 있다.

반영한 점 : 손님 주문받기 화면(⑥)을 OKPOS 후불 주문 방식으로 만들었다. 테이블을 고르고, 분류 버튼과 메뉴 버튼으로 메뉴를 담은 뒤 [주문]을 누르면 고른 메뉴들이 한 주문(orders + order_detail)으로 묶여 저장된다. 주문 내역 표에서 [선택 취소]·[전체 취소]와 숫자 키패드 수량 변경을 쓸 수 있다. 테이블 현황(⑤)에서 입장할 때 인원을 입력받는 것도 OKPOS 의 "고객 수 입력"을 참고했다.

다른 점 : OKPOS 는 음식값만 계산하지만, 보드게임 카페는 이용 시간 요금이 따로 있다. 그래서 입장 시각과 인원으로 이용료를 자동 계산하고, 나갈 때 음식값과 합쳐 한 번에 결제하며 이용료도 결제 상세에 한 줄로 남긴다. 테이블 이동·합석은 기능이 너무 많아져서 이번에는 넣지 않았다.

위아즈 보드게임 카페 솔루션

조사 내용 : 보드게임 카페 전용 프로그램으로, POS 와 테이블에 놓는 태블릿이 하나로 연결되어 있다. 이용 중인 테이블에는 남은 시간, 주문한 메뉴, 이용 시간 막대, 결제할 금액이 표시된다. 손님이 태블릿에서 직접 음식을 주문하면 POS 에 바로 들어가서 직원이 테이블까지 가지 않아도 된다. 매장에 있는 게임을 인원 수·장르·난이도로 나눠서 검색할 수 있고, 매장이 추천하는 게임도 따로 보여 준다.

반영한 점 : 테이블 현황(⑤)에서 테이블 카드에 인원과 이용 시간을 보여 주는 화면을 참고했다. 손님용 키오스크(③⑥)도 같은 구조로 만들었다. 손님이 직접 주문하면 POS 에 바로 들어가고(5 초마다 새 주문 알림), 인원·난이도·플레이 시간·장르로 게임을 추천받을 수 있다. 키오스크 이용 현황(⑥)에서 지금까지의 이용 시간과 예상 요금을 확인하는 기능도 여기서 가져왔다.

다른 점 : 위아즈는 이용권을 사고 남은 시간을 보여 주는 방식이 중심이다. 우리는 쓴 시간만큼 나중에 내는 후불제라서 남은 시간 대신 경과 시간과 예상 요금을 보여 준다. 추천할 때 부품이 빠졌거나 재고가 없는 게임은 빠지도록 게임 상태·재고와 연결했고, 원하는 게임이 모두 대여 중이면 키오스크에서 예약하고 반납되면 POS 에서 다음 예약 테이블을 호출하는 기능을 넣었다. 또 위아즈에는 없는 직원 근무·급여 관리, 발주(입고 시 재고 증가), 일별·월별 매출 집계를 관리자 화면에 넣었다.